

视觉2 (Visual Odometry)

王永才

2D-2D对极几何

- 对极几何
- 单应矩阵

三角化

3D-2D:PnP

3D-3D:ICP

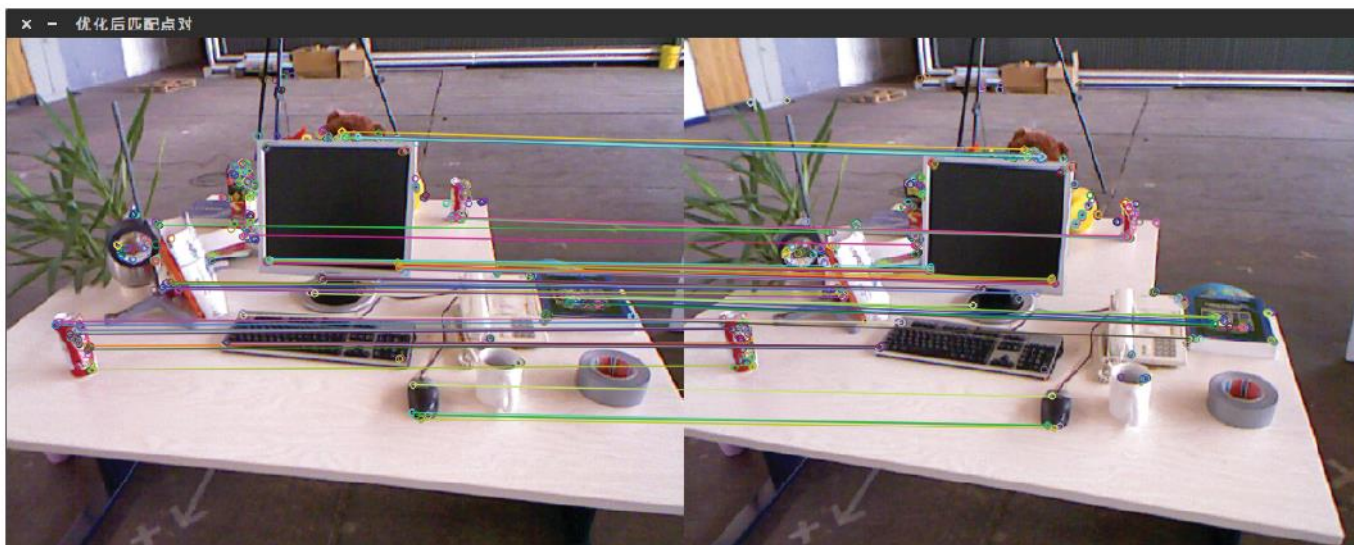
光流法

直接法



回顾上节课内容

- 特征匹配
 - 通过描述子的差异判断哪些特征为同一个点
 - 暴力匹配：比较图1中每个特征和图2特征的距离
 - 加速：快速最近邻（FLANN）

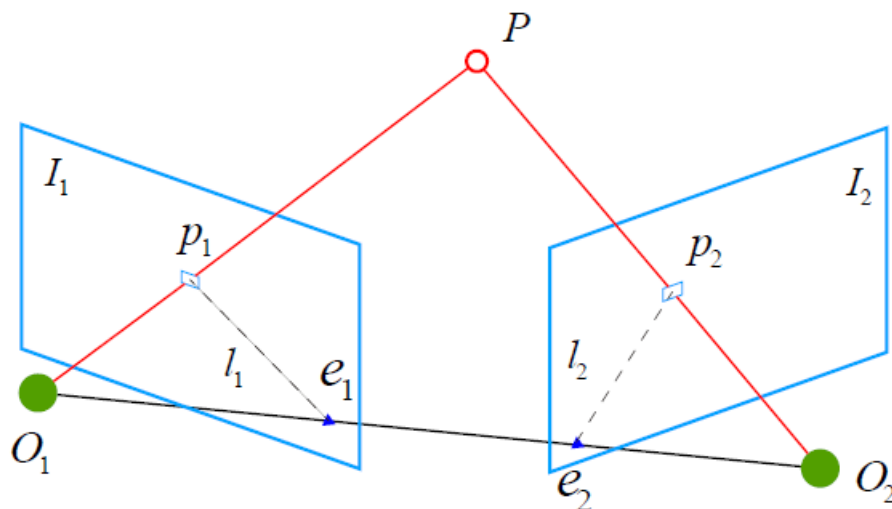


2D-2D对极几何

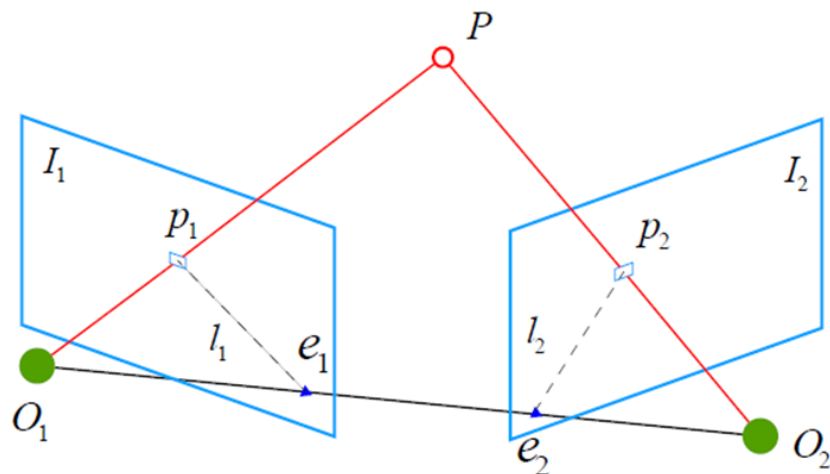
- 特征匹配之后，得到了特征点之间的对应关系
 - 如果只有两个单目图像，得到2D-2D间的关系
 - 如果匹配的是帧和地图，得到3D-2D间的关系
 - 如果匹配的是RGB-D图，得到3D-3D间的关系

——对极几何
——PnP
——ICP

对极几何：得到相机在两个不同位置上拍摄的同一对象的两幅照片，计算相机的位姿变换



对极约束



对极约束：描述 O_1 , O_2 , P 在同一个平面上

$$x_2^T t^\wedge R x_1 = 0. \quad p_2^T K^{-T} t^\wedge R K^{-1} p_1 = 0.$$

定义：

$$E = t^\wedge R, \quad F = K^{-T} E K^{-1}$$

Essential矩阵 Fundamental矩阵

$$x_2^T E x_1 = p_2^T F p_1 = 0.$$

这里注意对极关系是
对任何尺度成立的

2D-2D对极几何--八点法

八对点构成方程组

$$\begin{pmatrix} u_1^1 u_2^1 & u_1^1 v_2^1 & u_1^1 & v_1^1 u_2^1 & v_1^1 v_2^1 & v_1^1 & u_2^1 & v_2^1 & 1 \\ u_1^2 u_2^2 & u_1^2 v_2^2 & u_1^2 & v_1^2 u_2^2 & v_1^2 v_2^2 & v_1^2 & u_2^2 & v_2^2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_1^8 u_2^8 & u_1^8 v_2^8 & u_1^8 & v_1^8 u_2^8 & v_1^8 v_2^8 & v_1^8 & u_2^8 & v_2^8 & 1 \end{pmatrix} \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \\ e_6 \\ e_7 \\ e_8 \\ e_9 \end{pmatrix} = 0.$$

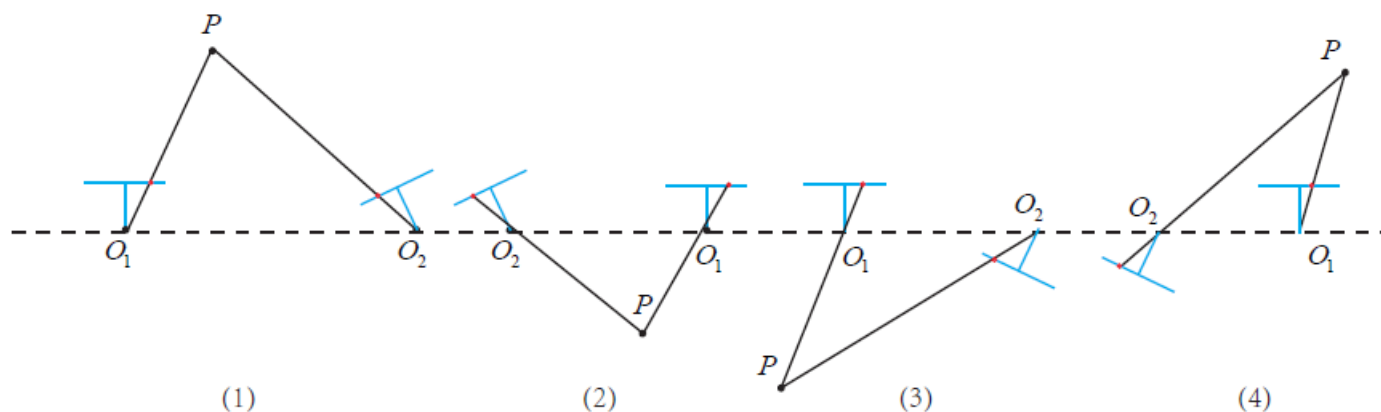
求解线性方程组得到 F .

2D-2D对极几何--八点法

- 从 E 计算 R, t : 奇异值分解

$$E = U\Sigma V^T, \quad \begin{aligned} t_1^\wedge &= UR_Z(\frac{\pi}{2})\Sigma U^T, & R_1 &= UR_Z^T(\frac{\pi}{2})V^T \\ t_2^\wedge &= UR_Z(-\frac{\pi}{2})\Sigma U^T, & R_2 &= UR_Z^T(-\frac{\pi}{2})V^T. \end{aligned}$$

四个可能的解，但只有一个深度为正



t 是受 E 的特征值大小影响的

2D-2D对极几何--八点法

- SVD过程中：

- 取 $E = U \text{diag}(\frac{\sigma_1 + \sigma_2}{2}, \frac{\sigma_1 + \sigma_2}{2}, 0) V^T$. 因为E的内在性质要求它的奇异值为

$$\sigma, \sigma, 0$$

- 最少可使用五个点计算R,t, 称为五点法
 - 但需要利用E的非线性性质，原理较复杂
- 多于八对点时：最小二乘或RANSAC
- OpenCV中findFundamentalMat(points1, points2, CV_FM_8POINT)
- findEssentialMat(points1, points2, focal_length, principal_point)

2D-2D对极几何——单应矩阵

- 从单应矩阵恢复 R, t :

- 八点法在特征点共面时会退化 (例如航拍, 从空中拍摄地面)

- 设特征点位于某平面上: $n^T P + d = 0$. 或 $-\frac{n^T P}{d} = 1$. (平面的表示)

- 两个图像特征点的坐标关系:

$$p_1 = KP$$

$$p_2 = K(RP + t)$$

$$= K \left(RP + t \cdot \left(-\frac{n^T P}{d} \right) \right)$$

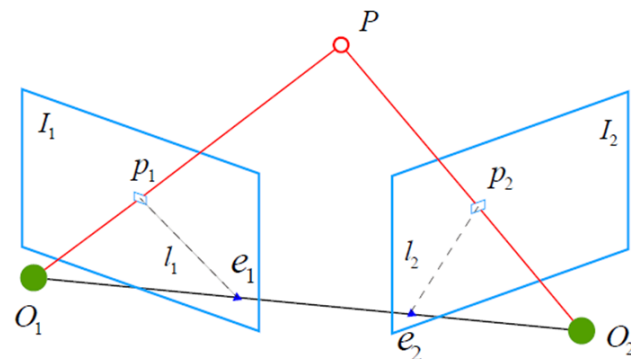
$$= K \left(R - \frac{tn^T}{d} \right) P$$

$$= K \left(R - \frac{tn^T}{d} \right) K^{-1} p_1.$$

中间记为H

$$p_2 = H p_1.$$

$$\begin{pmatrix} u_2 \\ v_2 \\ 1 \end{pmatrix} = \begin{pmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{pmatrix} \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix}.$$



2D-2D对极几何——单应矩阵

- $$\begin{pmatrix} u_2 \\ v_2 \\ 1 \end{pmatrix} = \begin{pmatrix} h_1 & h_2 & h_3 \\ h_4 & h_5 & h_6 \\ h_7 & h_8 & h_9 \end{pmatrix} \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix}$$
- 去掉第三行：

$$u_2 = \frac{h_1 u_1 + h_2 v_1 + h_3}{h_7 u_1 + h_8 v_1 + h_9}$$

$$v_2 = \frac{h_4 u_1 + h_5 v_1 + h_6}{h_7 u_1 + h_8 v_1 + h_9}$$

- 一对点提供两个约束
- 写成关于H的线性方程：
- 四对点构成8个方程

$$\begin{pmatrix} u_1^1 & v_1^1 & 1 & 0 & 0 & 0 & -u_1^1 u_2^1 & -v_1^1 u_2^1 \\ 0 & 0 & 0 & u_1^1 & v_1^1 & 1 & -u_1^1 v_2^1 & -v_1^1 v_2^1 \\ u_1^2 & v_1^2 & 1 & 0 & 0 & 0 & -u_1^2 u_2^2 & -v_1^2 u_2^2 \\ 0 & 0 & 0 & u_1^2 & v_1^2 & 1 & -u_1^2 v_2^2 & -v_1^2 v_2^2 \\ u_1^3 & v_1^3 & 1 & 0 & 0 & 0 & -u_1^3 u_2^3 & -v_1^3 u_2^3 \\ 0 & 0 & 0 & u_1^3 & v_1^3 & 1 & -u_1^3 v_2^3 & -v_1^3 v_2^3 \\ u_1^4 & v_1^4 & 1 & 0 & 0 & 0 & -u_1^4 u_2^4 & -v_1^4 u_2^4 \\ 0 & 0 & 0 & u_1^4 & v_1^4 & 1 & -u_1^4 v_2^4 & -v_1^4 v_2^4 \end{pmatrix} \begin{pmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \end{pmatrix} = \begin{pmatrix} u_2^1 \\ v_2^1 \\ u_2^2 \\ v_2^2 \\ u_2^3 \\ v_2^3 \\ u_2^4 \\ v_2^4 \end{pmatrix}$$

- 类似八点法

- 先计算H
- 再用H恢复R,t,n,d,K
- Opencv中 `findHomography(points1, points2, RANSAC, n);`

2D-2D对极几何——实践

```
Mat img_1 = imread ( argv[1], CV_LOAD_IMAGE_COLOR );
Mat img_2 = imread ( argv[2], CV_LOAD_IMAGE_COLOR );

vector<KeyPoint> keypoints_1, keypoints_2;
vector<DMatch> matches;
find_feature_matches ( img_1, img_2, keypoints_1, keypoints_2, matches );
```



```
Mat descriptors_1, descriptors_2;
// used in OpenCV3
Ptr<FeatureDetector> detector = ORB::create();
Ptr<DescriptorExtractor> descriptor = ORB::create();
Ptr<DescriptorMatcher> matcher = DescriptorMatcher::create ( "BruteForce-Hamming" );
//-- 第一步:检测 Oriented FAST 角点位置
detector->detect ( img_1, keypoints_1 );
detector->detect ( img_2, keypoints_2 );

//-- 第二步:根据角点位置计算 BRIEF 描述子
descriptor->compute ( img_1, keypoints_1, descriptors_1 );
descriptor->compute ( img_2, keypoints_2, descriptors_2 );

//-- 第三步:对两幅图像中的BRIEF描述子进行匹配, 使用 Hamming 距离
vector<DMatch> match;
//BFMatcher matcher ( NORM_HAMMING );
matcher->match ( descriptors_1, descriptors_2, match );
```

2D-2D对极几何——实践

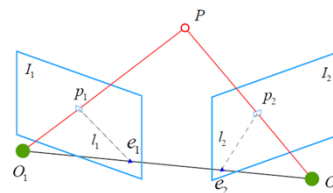
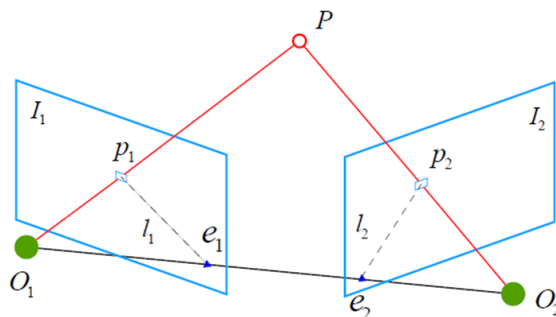
```
//-- 估计两张图像间运动
Mat R,t;
pose_estimation_2d2d ( keypoints_1, keypoints_2, matches, R, t );

for ( int i = 0; i < ( int ) matches.size(); i++ )
{
    points1.push_back ( keypoints_1[matches[i].queryIdx].pt );
    points2.push_back ( keypoints_2[matches[i].trainIdx].pt );
}

//-- 计算基础矩阵
Mat fundamental_matrix;
fundamental_matrix = findFundamentalMat ( points1, points2, CV_FM_8POINT );
cout<<"fundamental_matrix is "<<endl<< fundamental_matrix<<endl;

//-- 计算本质矩阵
Point2d principal_point ( 325.1, 249.7 ); //相机光心, TUM dataset标定值
double focal_length = 521; //相机焦距, TUM dataset标定值
Mat essential_matrix;
essential_matrix = findEssentialMat ( points1, points2, focal_length, principal_point );
cout<<"essential_matrix is "<<endl<< essential_matrix<<endl;
```

小结



- 2D-2D情况下，只知道图像坐标之间的对应关系
 - 通常采用8点法使用 E 或 F ，求相机的位姿变化 R, t
 - 这里注意
 - 对极约束要求相机有平移才能满足。
 - 对极约束忽略了尺度信息，具有尺度等价性
 - 当特征点在平面上时（例如俯视或仰视），使用单应矩阵 H 恢复 R, t
 - t 的尺度不一定是真实尺度
- 求得 R, t 后：
 - 可以利用三角化计算特征点的3D位置（即深度），恢复的3D坐标是受 t 的尺度影响的。

2D-2D对极几何

- 对极几何
- 单应矩阵



三角化

3D-2D:PnP

3D-3D:ICP

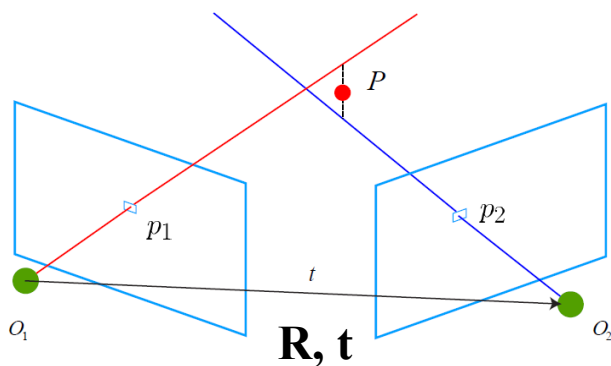
光流法

直接法



三角化-建图的基础

已知特征点在两幅图像中的像素坐标，并且已知相机运动 $\mathbf{R}$, $\mathbf{t}$ ，求解特征点的3D位置。



已知 p_1, p_2, R_{21}, t_{21}

求： P 的3D坐标

根据对极几何求出 $\mathbf{R}, \mathbf{t}$ 之后， P 的世界坐标，我们仍然是不知道的

回顾相机方程

$$Zp_1 = Z \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} = KP_1$$

$$Z_2p_2 = Z_2 \begin{pmatrix} u_2 \\ v_2 \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X_2 \\ Y_2 \\ Z_2 \end{pmatrix} = KP_2 = K(R_{21}P_1 + t_{21})$$

三角化-建图的基础

在相机1的拍摄图像中

$$Z_i \begin{pmatrix} u_i \\ v_i \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X_i \\ Y_i \\ Z_i \end{pmatrix} = K \mathbf{X}_i$$

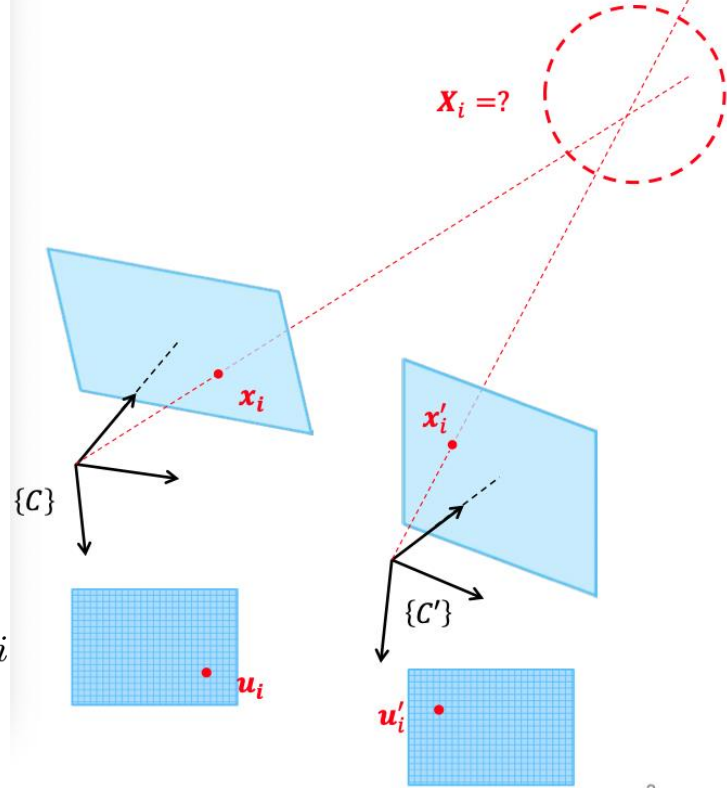
$$\begin{pmatrix} u_i \\ v_i \\ 1 \end{pmatrix} = \frac{1}{Z_i} \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X_i \\ Y_i \\ Z_i \end{pmatrix} = \frac{1}{Z_i} K \mathbf{X}_i$$

写成 $\mathbf{u}_i = s_i P \mathbf{X}_i$ 其中 $s_i = \frac{1}{Z_i}$, $P = [K, 0]$, $\mathbf{X}_i$ 是齐次坐标

$$\mathbf{u}_i = s_i P \mathbf{X}_i = s_i \begin{pmatrix} p_1 & p_2 & p_3 & p_4 \\ p_5 & p_6 & p_7 & p_8 \\ p_9 & p_{10} & p_{11} & p_{12} \end{pmatrix} \mathbf{X}_i$$

P 已知, s_i 未知, 但我们可以用叉乘的性质消掉它

$$\mathbf{u}_i \times s_i P \mathbf{X}_i = 0 \quad \rightarrow \quad \mathbf{u}_i \times P \mathbf{X}_i = 0 \quad \rightarrow \quad \begin{pmatrix} 0 & -1 & v_i \\ 1 & 0 & -u_i \\ -v_i & u_i & 0 \end{pmatrix} \begin{pmatrix} \mathbf{p}^1 \\ \mathbf{p}^2 \\ \mathbf{p}^3 \end{pmatrix} \mathbf{X}_i = 0$$



同理，在相机2的拍摄图像中

$$\begin{pmatrix} u'_i \\ v'_i \\ 1 \end{pmatrix} = \frac{1}{Z'_i} \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} T_{cw} \begin{pmatrix} X_i \\ Y_i \\ Z_i \end{pmatrix} = \frac{1}{Z'_i} K T_{cw} \mathbf{X}_i$$

$$\mathbf{u}'_i = s'_i P' \mathbf{X}_i = s'_i \begin{pmatrix} p'_1 & p'_2 & p'_3 & p'_4 \\ p'_5 & p'_6 & p'_7 & p'_8 \\ p'_9 & p'_{10} & p'_{11} & p'_{12} \end{pmatrix} \mathbf{X}_i$$

其中 $s'_i = \frac{1}{Z'_i}$, $P' = [K, 0] T_{cw, 3 \times 4}$, $\mathbf{X}_i$ 是齐次坐标

同样利用叉乘的性质，消去 s'_i

$$\mathbf{u}'_i \times s'_i P' \mathbf{X}_i = 0$$



$$\mathbf{u}'_i \times P' \mathbf{X}_i = 0$$

$$\begin{pmatrix} 0 & -1 & v'_i \\ 1 & 0 & -u'_i \\ -v'_i & u'_i & 0 \end{pmatrix} \begin{pmatrix} \mathbf{p}'^1 \\ \mathbf{p}'^2 \\ \mathbf{p}'^3 \end{pmatrix} \mathbf{X}_i = 0$$

三角化-直接线性变换 Direct Linear Transform(DLT)

相机1

$$\begin{pmatrix} 0 & -1 & v_i \\ 1 & 0 & -u_i \\ -v_i & u_i & 0 \end{pmatrix} \begin{pmatrix} \mathbf{p}^1 \\ \mathbf{p}^2 \\ \mathbf{p}^3 \end{pmatrix} \mathbf{X}_i = 0$$



$$\begin{pmatrix} -\mathbf{p}^2 + v_i \mathbf{p}^3 \\ \mathbf{p}^1 - u_i \mathbf{p}^3 \\ -v_i \mathbf{p}^1 + u_i \mathbf{p}^2 \end{pmatrix} \mathbf{X}_i = 0$$



$$\begin{pmatrix} -\mathbf{p}^2 + v_i \mathbf{p}^3 \\ \mathbf{p}^1 - u_i \mathbf{p}^3 \end{pmatrix} \mathbf{X}_i = 0$$

取出2个独立方程

相机2

$$\begin{pmatrix} 0 & -1 & v'_i \\ 1 & 0 & -u'_i \\ -v'_i & u'_i & 0 \end{pmatrix} \begin{pmatrix} \mathbf{p}'^1 \\ \mathbf{p}'^2 \\ \mathbf{p}'^3 \end{pmatrix} \mathbf{X}_i = 0$$



$$\begin{pmatrix} -\mathbf{p}'^2 + v'_i \mathbf{p}'^3 \\ \mathbf{p}'^1 - u'_i \mathbf{p}'^3 \end{pmatrix} \mathbf{X}_i = 0$$

取出2个独立方程



$$\begin{pmatrix} -\mathbf{p}^2 + v_i \mathbf{p}^3 \\ \mathbf{p}^1 - u_i \mathbf{p}^3 \\ -\mathbf{p}'^2 + v'_i \mathbf{p}'^3 \\ \mathbf{p}'^1 - u'_i \mathbf{p}'^3 \end{pmatrix} \mathbf{X}_i = 0$$

共同建立起4个
方程，求解4个
未知数

三角化-直接线性变换 Direct Linear Transform(DLT)

$$\begin{pmatrix} -\mathbf{p}^2 + v_i \mathbf{p}^3 \\ \mathbf{p}^1 - u_i \mathbf{p}^3 \\ -\mathbf{p}'^2 + v'_i \mathbf{p}'^3 \\ \mathbf{p}'^1 - u'_i \mathbf{p}'^3 \end{pmatrix} \mathbf{X}_i = 0 \quad \text{有两个以上的图像观测，即可求出}\mathbf{X}_i$$

首先： $A^T A X = 0$ 的解即是 $A X = 0$ 的解

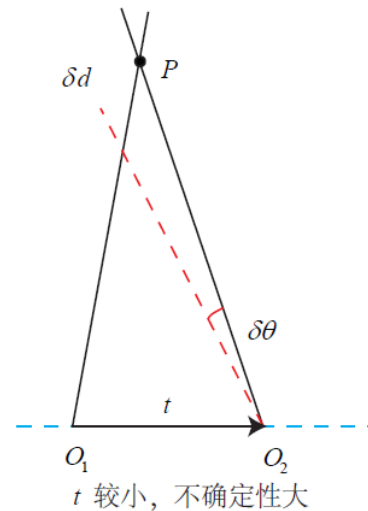
齐次线性方程组有非零解的充要条件是它的系数行列式 $|A^T A| = 0$ ，否则只有唯一零解，上式有无穷多解

使用SVD求最小二乘解， $A^T A$ 的最小特征值(0)，对应的特征向量，即是 $A X_i = 0$ 的解。解可能不满足齐次坐标形式（第四个元素为1），因此

$$\mathbf{X}_i = \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x_0/x_3 \\ x_1/x_3 \\ x_2/x_3 \\ x_3/x_3 \end{pmatrix} \quad \text{转换为齐次坐标}$$

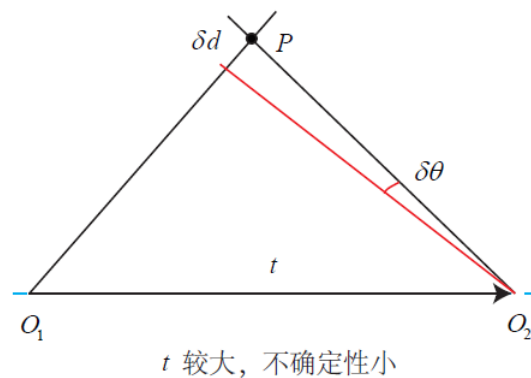
- 三角化中的问题：

- 有平移时才能进行三角测量（为什么？）
- 但是平移大时特征匹配可能不成功



- 相机前进时，虽然有位移，但位于图像中心的点无法三角化（没有视差）

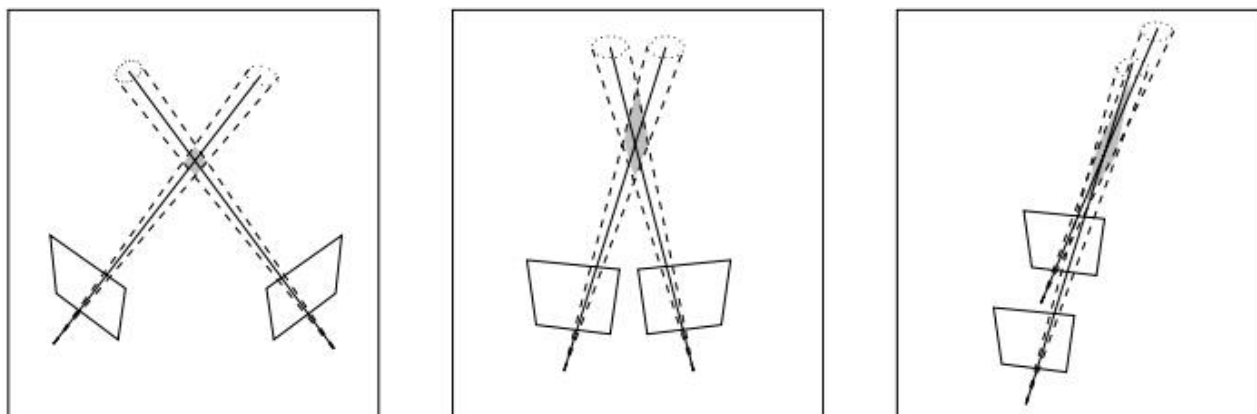
- 三角化之后的相对结构是正确的，但尺度信息不一定是正确的



直接线性变换三角测量的问题

P三维坐标的推导过程中，可以看到和**本征矩阵E**是息息相关的，**E和尺度**紧密相连，所以计算出来的**深度**

和根是

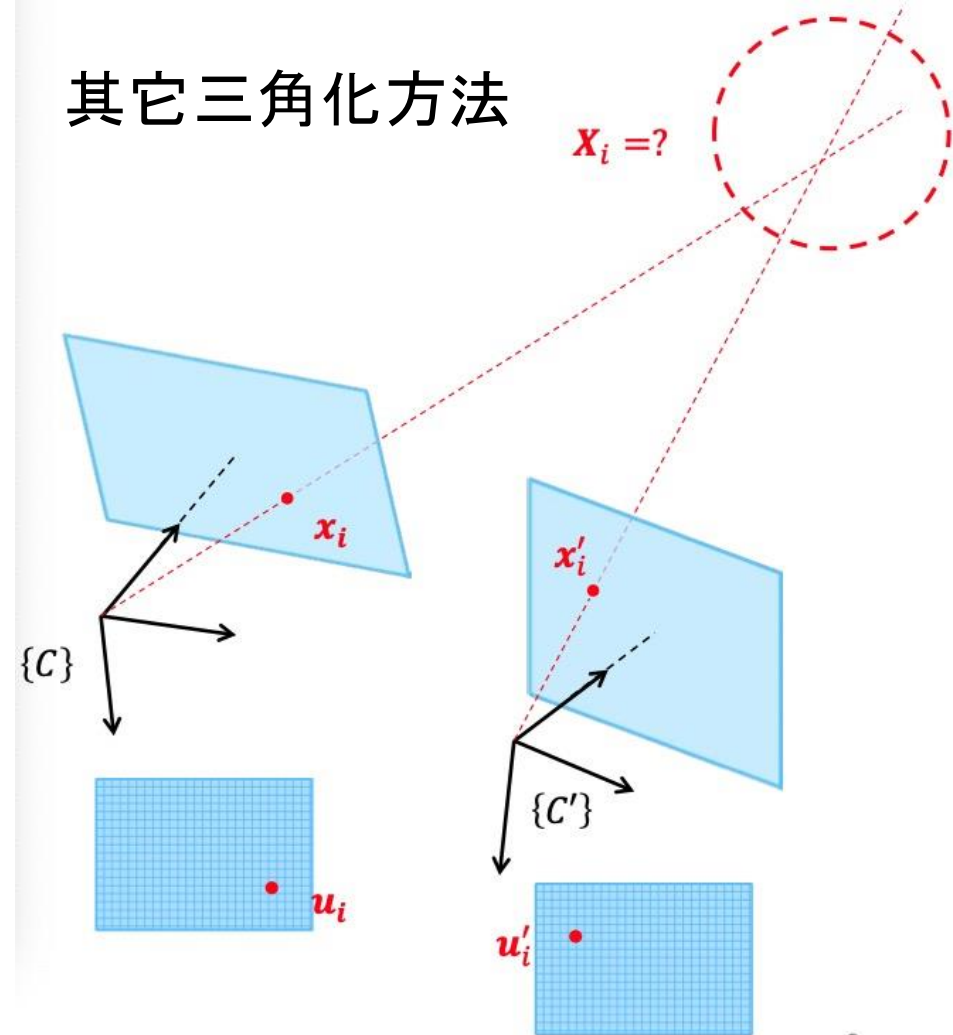


：实。
。

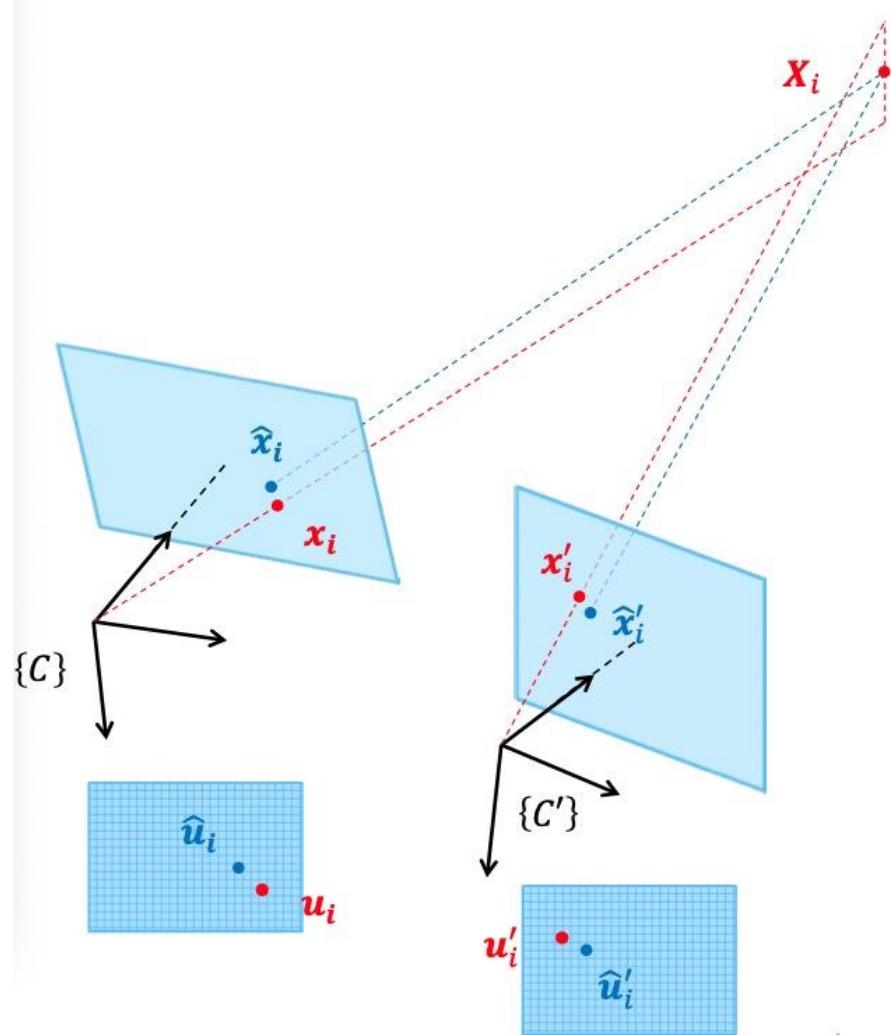
Fig. 12.6. **Uncertainty of reconstruction.** The shaded region in each case illustrates the shape of the uncertainty region, which depends on the angle between the rays. Points are less precisely localized along the ray as the rays become more parallel. Forward motion in particular can give poor reconstructions since rays are almost parallel for much of the field of view.

其它三角化方法

$X_i = ?$



由于图上像素是离散的，且观测有噪声， O_1p_1 , O_2p_2 可能不会交于一点，一个直观的想法是求两个直线距离最近的线段的中点，称为mid-point方法。



$$\epsilon_i = d(u_i, PX_i)^2 + d(u'_i, P'X_i)^2$$

$$\epsilon_i = d(u_i, \hat{u}_i)^2 + d(u'_i, \hat{u}'_i)^2$$

求使得投影误差之和最小的 X_i

需要求解优化方程

三角化总结

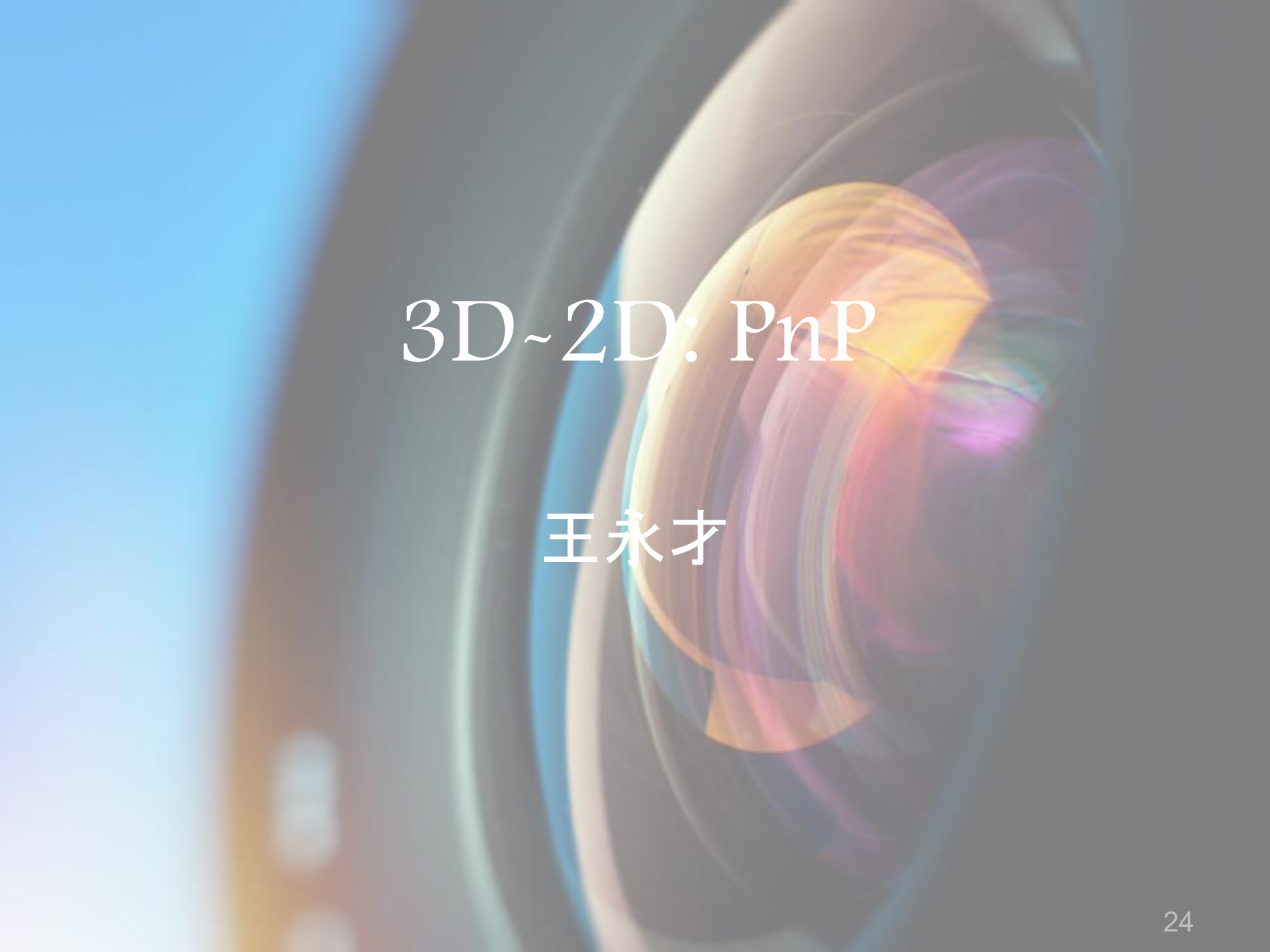
三角化是稀疏点建图的基础。

但是基于2D对极几何三角化算出来的点的空间坐标是不准确的。

并且要十分注意，每两对图像计算E并恢复R,t时，他们的尺度都不是一样，本来是同一点，在不同尺寸下，深度不一样了，这时候mapping最头痛了，所以这个尺度需要统一。

解决方法：后续的位姿估计我不用这种2d-2d计算本征E的方式了，有了场景点的3D坐标，后续的视频序列就可以通过3Dto2D(solvePnp)来进行位姿估计。

这样后续的视频序列就不用计算本征矩阵从而绕过了尺度，所以后续的视频序列的位姿和最开始的两帧的尺度是一样的了。

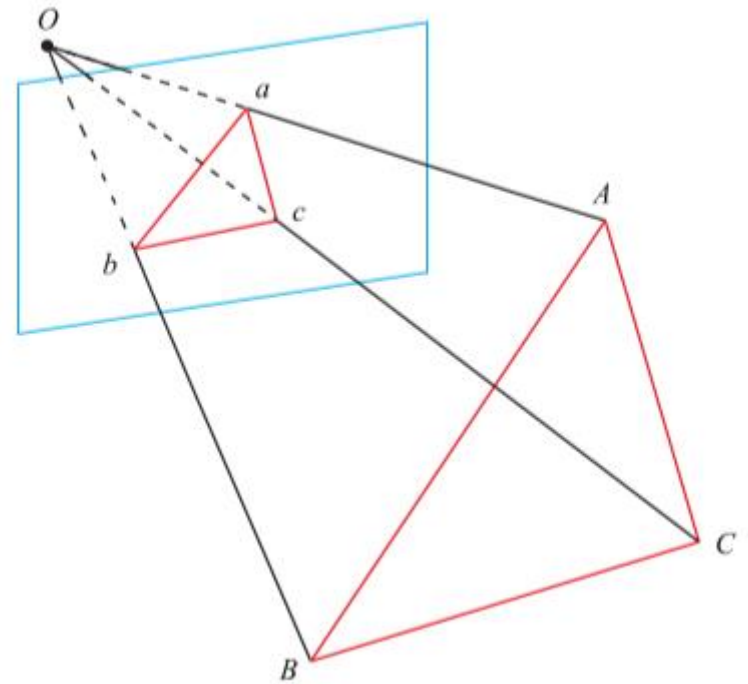


3D~2D: PnP

王永才

3D-2D: PnP

- 已知3D点的空间位置和在两个相机上的投影点坐标，求相机的旋转和平移（外参）
- 在RGB-D，双目视觉里程计中常使用
- 代数的解法/优化的解法
- 代数的
 - DLT
 - P3P
 - EPnP/UPnP/...
- 优化的：Bundle Adjustment



DLT: 直接线性变换求解PnP

- DLT

- 设空间点 $P_1 = (X, Y, Z, 1)^T$ 相机1坐标系下的3D坐标
- 在相机2坐标 $P_2 = (X_2, Y_2, Z_2, 1)^T$
- 在相机2的像素坐标 $u = (u, v, 1)^T$
- 投影到 $z=1$ 的投影坐标 $x_2 = (X_2/Z_2, Y_2/Z_2, 1)^T$

- 投影关系： $Z_2 u = K P_2 = K T_{21} P_1$

- 展开： $s K^{-1} u = P_2 = T_{21} P_1$

相机2坐标系坐标
= T_{21} *相机1坐标系下的
的3D坐标

- $$s \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix} = \begin{pmatrix} t_1 & t_2 & t_3 & t_4 \\ t_5 & t_6 & t_7 & t_8 \\ t_9 & t_{10} & t_{11} & t_{12} \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}.$$

$$s u = K P_2 = K T_{21} P_1$$

P_1 已知, 求 T_{21}

DLT: 直接线性变换求解PnP

$$s \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix} = \begin{pmatrix} t_1 & t_2 & t_3 & t_4 \\ t_5 & t_6 & t_7 & t_8 \\ t_9 & t_{10} & t_{11} & t_{12} \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}.$$

- 注意最下一行为

$$s = [t_9, t_{10}, t_{11}, t_{12}][X, Y, Z, 1]^T$$

- 用它消掉前两行中的s, 则一个特征点提供两个方程：

$$\mathbf{t}_1 = (t_1, t_2, t_3, t_4)^T, \mathbf{t}_2 = (t_5, t_6, t_7, t_8)^T, \mathbf{t}_3 = (t_9, t_{10}, t_{11}, t_{12})^T,$$

$$t_1^T P - t_3^T P u_1 = 0$$

$$t_2^T P - t_3^T P v_1 = 0$$

- 为求解12个未知数, 需要12/2=6对点。(超定时求最小二乘解)

DLT: 直接线性变换求解PnP

- DLT将 $\mathbf{R}, \mathbf{t}$ 看成独立的未知量，所以在求出结果后，需要将 $\mathbf{t}$ 组成的矩阵投影回 $\text{SO}(3)$
- 此外，也可代入内参矩阵 $\mathbf{K}$ ，但SLAM中一般假设 $\mathbf{K}$ 已知，所以这里没有代入。

$$s \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix} = \begin{pmatrix} t_1 & t_2 & t_3 & t_4 \\ t_5 & t_6 & t_7 & t_8 \\ t_9 & t_{10} & t_{11} & t_{12} \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}.$$

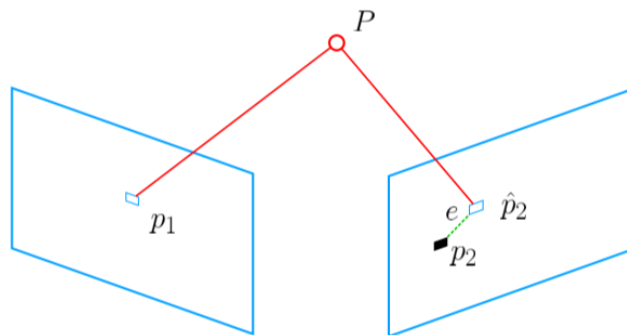

$$\mathbf{R} \leftarrow (\mathbf{R}\mathbf{R}^T)^{-\frac{1}{2}} \mathbf{R}$$

PnP 的优化解法

- PnP的优化解法：Bundle Adjustment
 - 最小化重投影误差 (Minimizing a reprojection error)
 - 投影关系 $s_i \mathbf{u}_i = \mathbf{K} \exp(\boldsymbol{\xi}^\wedge) \mathbf{P}_i$.
 - 定义重投影误差并取最小化：

$$\boldsymbol{\xi}^* = \arg \min_{\boldsymbol{\xi}} \frac{1}{2} \sum_{i=1}^n \left\| \mathbf{u}_i - \frac{1}{s_i} \mathbf{K} \exp(\boldsymbol{\xi}^\wedge) \mathbf{P}_i \right\|_2^2.$$

$\mathbf{P}$: 3d点, $\boldsymbol{\xi}$ 相机位姿; $\mathbf{K}$ 内参; $\mathbf{u}$ 投影



PnP 的优化解法

- 线性化和雅可比

- 考虑单个投影点误差： $e_i = u_i - \frac{1}{s_i} K \exp(\xi) P_i$
- 线性化： $e_i(\xi + \Delta\xi) \approx e_i(\xi) + J(\xi)\Delta\xi$
- 雅可比的形式？

$$\frac{\partial e}{\partial \delta \xi} = \lim_{\delta \xi \rightarrow 0} \frac{e(\delta \xi \oplus \xi)}{\delta \xi} = \frac{\partial e}{\partial P'} \frac{\partial P'}{\partial \delta \xi}. \quad \text{其中 } P' = TP$$

第一项：
$$\frac{\partial e}{\partial P'_i} = \frac{\partial(u_i - \frac{1}{s_i} K P'_i)}{\partial P'_i} = 0 - \frac{\partial(\frac{1}{s_i} K P'_i)}{\partial P'_i} \quad (1)$$

由于 $su = KP'$  $\frac{\partial(\frac{1}{s_i} K P'_i)}{\partial P'_i} = \frac{\partial u_i}{\partial P'_i}$

而由于
$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}.$$

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}.$$

$$u = f_x \frac{X'}{Z'} + c_x, \quad v = f_y \frac{Y'}{Z'} + c_y.$$

第一项有：

$$\begin{aligned} \frac{\partial \mathbf{e}}{\partial \mathbf{P}'} &= - \begin{bmatrix} \frac{\partial u}{\partial X'} & \frac{\partial u}{\partial Y'} & \frac{\partial u}{\partial Z'} \\ \frac{\partial v}{\partial X'} & \frac{\partial v}{\partial Y'} & \frac{\partial v}{\partial Z'} \end{bmatrix} \\ &= - \begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} \end{bmatrix}. \end{aligned}$$

第二项

$$\frac{\partial e}{\partial \delta \xi} = \lim_{\delta \xi \rightarrow 0} \frac{e(\delta \xi \oplus \xi)}{\delta \xi} = \frac{\partial e}{\partial P'} \frac{\partial P'}{\partial \delta \xi}.$$

第二项：

$$\frac{\partial (TP)}{\partial \delta \xi} = (TP)^{\odot} = \begin{bmatrix} I & -P'^{\wedge} \\ \mathbf{0}^T & \mathbf{0}^T \end{bmatrix}.$$

• 非齐次形式：

$$\frac{\partial P'}{\partial \delta \xi} = [I, -P'^{\wedge}].$$

• 两项相乘：

$$\frac{\partial e}{\partial \delta \xi} = - \begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} & -\frac{f_x X' Y'}{Z'^2} & f_x + \frac{f_x X^2}{Z'^2} & -\frac{f_x Y'}{Z'} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} & -f_y - \frac{f_y Y'^2}{Z'^2} & \frac{f_y X' Y'}{Z'^2} & \frac{f_y X'}{Z'} \end{bmatrix}.$$

PnP的优化算法

$$\xi^* = \arg \min_{\xi} \frac{1}{2} \sum_{i=1}^n \left\| u_i - \frac{1}{s_i} K \exp(\xi) P_i \right\|_2^2.$$

for iter=0; iter<iterations; iter ++

for i=1... n **//基于所有点的测量求最优增量**

$$e_i = u_i - \frac{1}{s_i} K \exp(\xi) P_i$$

$$J_i = \frac{\partial e_i}{\partial \delta \xi} = - \begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} & -\frac{f_x X' Y'}{Z'^2} & f_x + \frac{f_x X^2}{Z'^2} & -\frac{f_x Y'}{Z'} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} & -f_y - \frac{f_y Y'^2}{Z'^2} & \frac{f_y X' Y'}{Z'^2} & \frac{f_y X'}{Z'} \end{bmatrix}.$$

$$H+ = J_i^T J_i$$

$$b+ = J_i^T e_i$$

endfor

$$\Delta \xi = -H^{-1} b$$

$$T_{est} = \exp(\Delta \xi) T_{est} \quad \text{//将最优增量应用在估计的变换矩阵上}$$

If Converge($\Delta \xi$) **//如果收敛或者达到最大迭代次数则退出**
 break;

endfor

code

PnP 的优化解法

- 除了位姿优化，我们还希望优化特征点的空间位置：

$$e_i = u_i - \frac{1}{s_i} K \exp(\xi^\wedge) P_i$$

$$\frac{\partial e}{\partial P} = \frac{\partial e}{\partial P'} \frac{\partial P'}{\partial P}$$

$$P' = (TP)_{1:3} = RP + t$$

$$\frac{\partial e}{\partial P} = - \begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} \end{bmatrix} R.$$

- 从而我们得到误差对于点的位置的导数。

用G2O实现PnP

- G2O是一个**通用的图优化**的工具，可以调用**G2O直接求解图优化问题**。
- 使用时，只要**定义好图中的顶点和边**即可，需要实现**雅克比矩阵的虚函数**，不需要自己写迭代过程。
- 在**g2o/types/sba/types_six_dof_expmap.h** 文件中，看一下G2O所提供的李代数表达的顶点和边。

相机位姿vertex模板类

李代数表示位姿的6维向量

```
class G2O_TYPES_SBA_API VertexSE3Expmap : public BaseVertex<6, SE3Quat>{  
public:
```

```
EIGEN_MAKE_ALIGNED_OPERATOR_NEW
```

```
VertexSE3Expmap();
```

//构造函数

```
bool read(std::istream& is);
```

```
bool write(std::ostream& os) const;
```

//外部读写

```
virtual void setToOriginImpl() {
```

```
    _estimate = SE3Quat();
```

```
}
```

//初始化虚函数，调用者可以实现

```
virtual void oplusImpl(const number_t* update_) {
```

```
    Eigen::Map<const Vector6> update(update_);
```

```
    setEstimate(SE3Quat::exp(update)*estimate());
```

```
}
```

//姿态更新虚函数

```
};
```

$$T_{est} = \exp(\Delta\xi)T_{est}$$

G2o定义的相机位姿

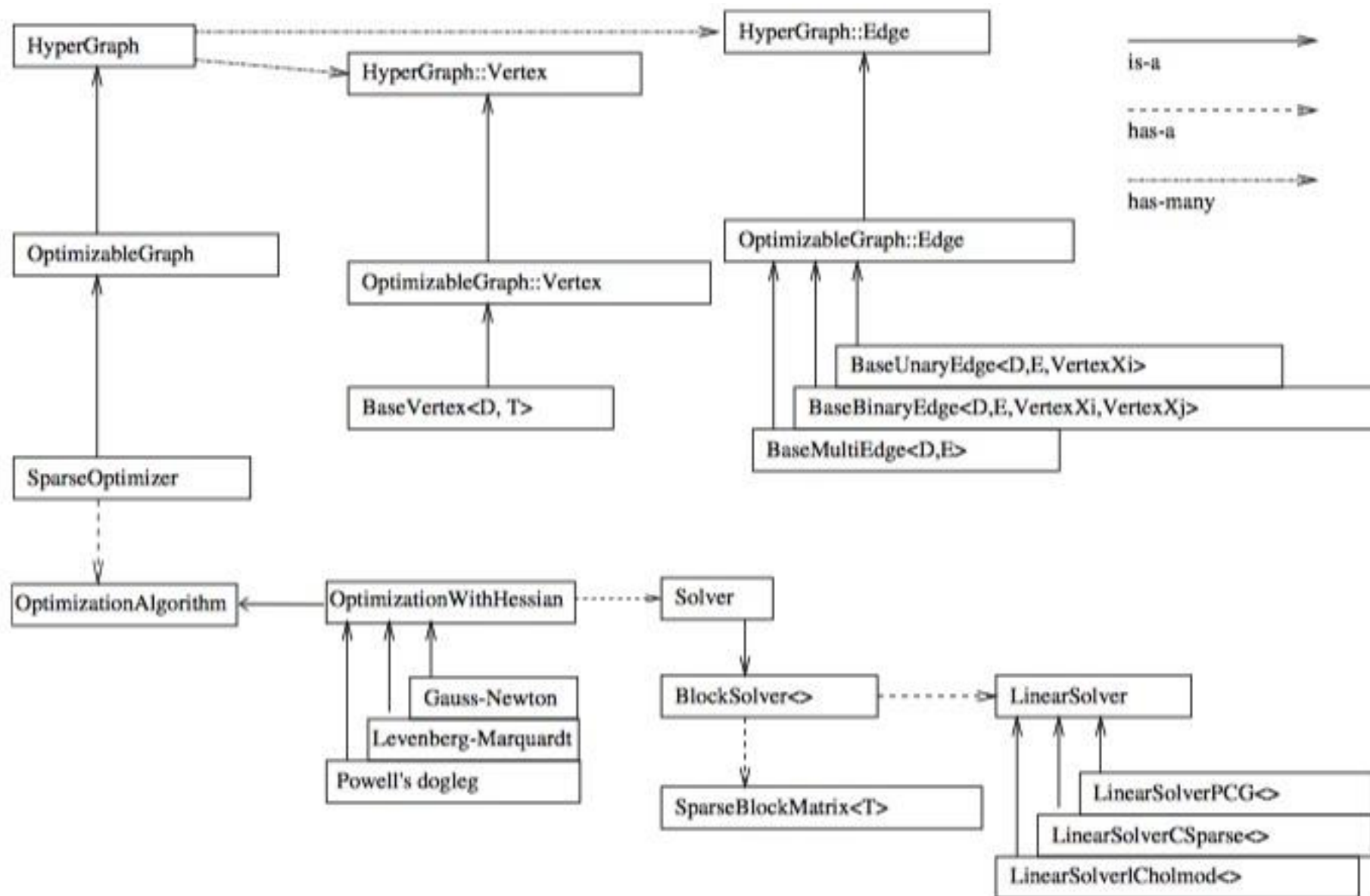


Figure 2: Class diagram of g2o.

3D空间点的Vertex模板类

```
class G2O_TYPES_SBA_API VertexSBAPointXYZ : public BaseVertex<3, Vector3>
{
    public:
        EIGEN_MAKE_ALIGNED_OPERATOR_NEW
        VertexSBAPointXYZ();
        virtual bool read(std::istream& is);
        virtual bool write(std::ostream& os) const;

        virtual void setToOriginImpl() {
            _estimate.fill(0);
        }

        virtual void oplusImpl(const number_t* update)
        {
            Eigen::Map<const Vector3> v(update);
            _estimate += v;
        }
};
```

//父类中3维向量存储x,y,z

//初始化的虚函数

//update的虚函数

由3D投影到2D坐标的投影误差边的模板类

```
class G2O_TYPES_SBA_API EdgeProjectXYZ2UV : public BaseBinaryEdge<2, Vector2, VertexSBAPointXYZ,
VertexSE3Expmap>{
public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW;

    EdgeProjectXYZ2UV();
    bool read(std::istream& is);
    bool write(std::ostream& os) const;

    void computeError() { //误差边残差计算
        const VertexSE3Expmap* v1 = static_cast<const VertexSE3Expmap*>(_vertices[1]); //相机位姿
        const VertexSBAPointXYZ* v2 = static_cast<const VertexSBAPointXYZ*>(_vertices[0]); //空间点x,y,z坐标
        const CameraParameters * cam
            = static_cast<const CameraParameters *>(parameter(0));
        Vector2 obs(_measurement); //相机观测像素坐标

        _error = obs - cam->cam_map(v1->estimate().map(v2->estimate()));
    } //相机观测像素坐标 - (x,y,z)投影到v1姿态的相机的相平面的投影坐标

    virtual void linearizeOplus();

    CameraParameters * _cam; //需要实现的误差边对位姿的雅克比矩阵
};
```

G2O中 EdgeProjectXYZ2UV类的虚函数的实现

```
void EdgeProjectXYZ2UV::computeError() {  
    const VertexSE3Expmap* v1 = static_cast<const VertexSE3Expmap*>(_vertices[1]);  
    //v1 是相机位姿  
  
    const VertexPointXYZ* v2 = static_cast<const VertexPointXYZ*>(_vertices[0]);  
  
    //v2是 x,y,z点  
  
    const CameraParameters* cam = static_cast<const  
CameraParameters*>(parameter(0));  
    //相机内参  
  
    _error = measurement() - cam->cam_map(v1->estimate()).map(v2->estimate());  
  
    //计算投影误差  
}
```

G2O中 EdgeProjectXYZ2UV类的虚函数的实现

```
void EdgeProjectXYZ2UV::linearizeOplus() {           //求雅克比矩阵
    VertexSE3Expmap* vj = static_cast<VertexSE3Expmap*>(_vertices[1]);

    SE3Quat T(vj->estimate());
    VertexPointXYZ* vi = static_cast<VertexPointXYZ*>(_vertices[0]);
    Vector3 xyz = vi->estimate();
    Vector3 xyz_trans = T.map(xyz);

    number_t x = xyz_trans[0];           //X', Y', Z'
    number_t y = xyz_trans[1];
    number_t z = xyz_trans[2];
    number_t z_2 = z * z;

    const CameraParameters* cam = static_cast<const
CameraParameters*>(parameter(0));

    Eigen::Matrix<number_t, 2, 3, Eigen::ColMajor> tmp;
    tmp(0, 0) = cam->focal_length;
    tmp(0, 1) = 0;
    tmp(0, 2) = -x / z * cam->focal_length;
```

接上一页

```
tmp(1, 0) = 0;  
tmp(1, 1) = cam->focal_length;  
tmp(1, 2) = -y / z * cam->focal_leng
```

$$\frac{\partial e}{\partial \mathbf{P}} = - \begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} \end{bmatrix} \mathbf{R}.$$

```
_jacobianOplusXi = -1. / z * tmp * T.rotation().toRotationMatrix();
```

```
_jacobianOplusXj(0, 0) = x * y / z_2 * cam->focal_length;  
_jacobianOplusXj(0, 1) = -(1 + (x * x / z_2)) * cam->focal_length;  
_jacobianOplusXj(0, 2) = y / z * cam->focal_length;  
_jacobianOplusXj(0, 3) = -1. / z * cam->focal_length;  
_jacobianOplusXj(0, 4) = 0;  
_jacobianOplusXj(0, 5) = x / z_2 * cam->focal_length;
```

```
_jacobianOplusXj(1, 0) = (1 + y * y / z_2) * cam->focal_length;  
_jacobianOplusXj(1, 1) = -x * y / z_2 * cam->focal_length;  
_jacobianOplusXj(1, 2) = -x / z * cam->focal_length;  
_jacobianOplusXj(1, 3) = 0;  
_jacobianOplusXj(1, 4) = -1. / z * cam->focal_length;  
_jacobianOplusXj(1, 5) = y / z_2 * cam->focal_length;
```

}

$$\frac{\partial e_i}{\partial \delta \xi} = - \begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} & -\frac{f_x X' Y'}{Z'^2} & f_x + \frac{f_x X^2}{Z'^2} & -\frac{f_x Y'}{Z'} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} & -f_y - \frac{f_y Y'^2}{Z'^2} & \frac{f_y X' Y'}{Z'^2} & \frac{f_y X'}{Z'} \end{bmatrix}.$$

PnP求解相机位姿实践

```
//-- 读取图像
```

```
Mat img_1 = imread(argv[1], CV_LOAD_IMAGE_COLOR);  
Mat img_2 = imread(argv[2], CV_LOAD_IMAGE_COLOR);
```

提取特征点并进行2D特征点匹配，找到点对

```
vector<KeyPoint> keypoints_1, keypoints_2;  
vector<DMatch> matches;  
find_feature_matches(img_1, img_2, keypoints_1, keypoints_2, matches);  
cout << "一共找到了" << matches.size() << "组匹配点" << endl;
```

```
// 建立3D点
```

```
Mat d1 = imread(argv[3], CV_LOAD_IMAGE_UNCHANGED); // 深度图为16位无符号数，单通道  
Mat K = (Mat_<double>(3, 3) << 520.9, 0, 325.1, 0, 521.0, 249.7, 0, 0, 1);  
vector<Point3f> pts_3d;  
vector<Point2f> pts_2d;  
for (DMatch m:matches) {  
    ushort d = d1.ptr<unsigned short>(int(keypoints_1[m.queryIdx].pt.y))[int(keypoints_1[m.trainIdx].pt.x)];  
    if (d == 0) // bad depth  
        continue;  
    float dd = d / 5000.0;  
    Point2d p1 = pixel2cam(keypoints_1[m.queryIdx].pt, K);  
    pts_3d.push_back(Point3f(p1.x * dd, p1.y * dd, dd));  
    pts_2d.push_back(keypoints_2[m.trainIdx].pt);  
}
```

```
Mat r, t;  
solvePnP(pts_3d, pts_2d, K, Mat(), r, t, false); // 调用OpenCV 的 PnP 求解  
Mat R;  
cv::Rodrigues(r, R); // r为旋转向量形式, 用Rodrigues公式转换为矩阵
```

```
//转换为Sophus库的2D, 3D点  
VecVector3d pts_3d_eigen;  
VecVector2d pts_2d_eigen;  
for (size_t i = 0; i < pts_3d.size(); ++i) {  
    pts_3d_eigen.push_back(Eigen::Vector3d(pts_3d[i].x, pts_3d[i].y, pts_3d[i].z));  
    pts_2d_eigen.push_back(Eigen::Vector2d(pts_2d[i].x, pts_2d[i].y));  
}
```

```
//定义要求的pose_gn是3D变换矩阵  
cout << "calling bundle adjustment by gauss newton" << endl;  
Sophus::SE3d pose_gn;  
t1 = chrono::steady_clock::now();  
bundleAdjustmentGaussNewton(pts_3d_eigen, pts_2d_eigen, K, pose_gn);
```

3D~3D: ICP

王永才

3D-3D: ICP

- 给定配对好的两组3D点，求其旋转和平移，可用迭代最近点（Iterative Closest Point, ICP求解）

- 设：
$$P = \{p_1, \dots, p_n\}, \quad P' = \{p'_1, \dots, p'_n\},$$

- 运动关系：
$$\forall i, p_i = Rp'_i + t.$$

- 同样定义误差项：

$$e_i = p_i - (Rp'_i + t).$$

- 以及最小二乘问题：

$$\min_{R, t} J = \frac{1}{2} \sum_{i=1}^n \|(p_i - (Rp'_i + t))\|_2^2.$$

3D-3D: ICP

$$\min_{\mathbf{R}, \mathbf{t}} J = \frac{1}{2} \sum_{i=1}^n \|\mathbf{p}_i - (\mathbf{R}\mathbf{p}'_i + \mathbf{t})\|_2^2.$$

1. 定义质心 : $\mathbf{p} = \frac{1}{n} \sum_{i=1}^n (\mathbf{p}_i), \quad \mathbf{p}' = \frac{1}{n} \sum_{i=1}^n (\mathbf{p}'_i).$

2. 改写目标函数 :

$$\begin{aligned} \frac{1}{2} \sum_{i=1}^n \|\mathbf{p}_i - (\mathbf{R}\mathbf{p}'_i + \mathbf{t})\|^2 &= \frac{1}{2} \sum_{i=1}^n \|\mathbf{p}_i - \mathbf{R}\mathbf{p}'_i - \mathbf{t} - \mathbf{p} + \mathbf{R}\mathbf{p}' + \mathbf{p} - \mathbf{R}\mathbf{p}'\|^2 \\ &= \frac{1}{2} \sum_{i=1}^n \|(\mathbf{p}_i - \mathbf{p} - \mathbf{R}(\mathbf{p}'_i - \mathbf{p}')) + (\mathbf{p} - \mathbf{R}\mathbf{p}' - \mathbf{t})\|^2 \\ &= \frac{1}{2} \sum_{i=1}^n (\|\mathbf{p}_i - \mathbf{p} - \mathbf{R}(\mathbf{p}'_i - \mathbf{p}'))\|^2 + \|\mathbf{p} - \mathbf{R}\mathbf{p}' - \mathbf{t}\|^2 + \end{aligned}$$

交叉项部分求和为零 $\Rightarrow 2(\mathbf{p}_i - \mathbf{p} - \mathbf{R}(\mathbf{p}'_i - \mathbf{p}'))^T (\mathbf{p} - \mathbf{R}\mathbf{p}' - \mathbf{t}).$

3D-3D: ICP

- 目标函数简化为：

$$\min_{R,t} J = \frac{1}{2} \sum_{i=1}^n \|p_i - p - R(p_i' - p')\|^2 + \|p - Rp' - t\|^2.$$

只和R有关

和R,t都有关

- 最小化第一项，然后取t，使得后一项为零。
- 左侧项如何求？

3D-3D: ICP

$$\min_{\mathbf{R}, \mathbf{t}} J = \frac{1}{2} \sum_{i=1}^n \|\mathbf{p}_i - \mathbf{p} - \mathbf{R}(\mathbf{p}_i' - \mathbf{p}')\|^2 + \|\mathbf{p} - \mathbf{R}\mathbf{p}' - \mathbf{t}\|^2.$$

- 旋转的求取
- 定义去质心坐标：

$$\mathbf{q}_i = \mathbf{p}_i - \mathbf{p}, \quad \mathbf{q}_i' = \mathbf{p}_i' - \mathbf{p}'.$$

- 最小化：

$$\mathbf{R}^* = \arg \min_{\mathbf{R}} \frac{1}{2} \sum_{i=1}^n \|\mathbf{q}_i - \mathbf{R}\mathbf{q}_i'\|^2.$$

- SVD解法：

$$\mathbf{W} = \sum_{i=1}^n \mathbf{q}_i \mathbf{q}_i'^T. \quad \mathbf{W} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T.$$

$$\mathbf{R} = \mathbf{U} \mathbf{V}^T.$$

SVD分解方法的推导过程

- 问题简介

- 考虑点集 $S=\{p_1, p_2, \dots, p_n\}$, 其重心为 c_s
- 考虑点集 $T=\{q_1, q_2, \dots, q_n\}$, 其重心为 c_T
- q_i 与 p_i 是相对应的点。

- 在经过重心对齐和旋转矩阵旋转之后, 变换后的 p_i 点为

$$p'_i = c_T + \mathbf{R} (p_i - c_s)$$

- 我们寻求旋转矩阵 $\mathbf{R}$, 使得所有点对的对齐误差之和最小:

$$\mathbf{R}^* = \arg \min_R \sum_{i=1:n} \|c_T + \mathbf{R} (p_i - c_s) - q_i\|^2$$

$$\begin{aligned} \mathbf{R}^* &= \arg \min_{\mathbf{R}} \sum_{i=1:n} \|\mathbf{R} (p_i - c_s) - (q_i - c_T)\| \\ &= \arg \min_{\mathbf{R}} \sum_{i=1:n} \|\mathbf{R} p_i - \mathbf{q}_i\| \end{aligned}$$

$$\mathbf{R}^* = \arg \min_{\mathbf{R}} \sum_{i=1:n} \|\mathbf{R}\mathbf{p}_i - \mathbf{q}_i\|^2 \Leftrightarrow \max tr(\mathbf{Q}^T \mathbf{R} \mathbf{P})$$

$$\begin{aligned} \|\mathbf{R}\mathbf{p}_i - \mathbf{q}_i\|^2 &= (\mathbf{R}\mathbf{p}_i - \mathbf{q}_i)^T (\mathbf{R}\mathbf{p}_i - \mathbf{q}_i) \\ &= (\mathbf{p}_i^T \mathbf{R}^T - \mathbf{q}_i^T) (\mathbf{R}\mathbf{p}_i - \mathbf{q}_i) \\ &= \mathbf{p}_i^T \mathbf{R}^T \mathbf{R} \mathbf{p}_i - \mathbf{p}_i^T \mathbf{R}^T \mathbf{q}_i - \mathbf{q}_i^T \mathbf{R} \mathbf{p}_i + \mathbf{q}_i^T \mathbf{q}_i \\ &= \mathbf{p}_i^T \mathbf{p}_i - \mathbf{p}_i^T \mathbf{R}^T \mathbf{q}_i - \mathbf{q}_i^T \mathbf{R} \mathbf{p}_i + \mathbf{q}_i^T \mathbf{q}_i \\ &= \mathbf{p}_i^T \mathbf{p}_i - 2\mathbf{q}_i^T \mathbf{R} \mathbf{p}_i + \mathbf{q}_i^T \mathbf{q}_i \end{aligned}$$

$$\min \sum_{i=1:n} \|\mathbf{R}\mathbf{p}_i - \mathbf{q}_i\|^2 \Leftrightarrow \max \sum_{i=1:n} \mathbf{q}_i^T \mathbf{R} \mathbf{p}_i$$

$$\mathbf{P} = [\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n]_{d \times n}, \mathbf{Q} = [\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_n]_{d \times n}$$

$$\mathbf{Q}^T \mathbf{R} \mathbf{P} = \begin{bmatrix} \mathbf{q}_1^T \\ \mathbf{q}_2^T \\ \vdots \\ \mathbf{q}_n^T \end{bmatrix} \mathbf{R} [\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_n] = \begin{bmatrix} \mathbf{q}_1^T \mathbf{R} \mathbf{p}_1 & \dots & \dots \\ \vdots & \mathbf{q}_2^T \mathbf{R} \mathbf{p}_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ \vdots & \dots & \dots & \mathbf{q}_n^T \mathbf{R} \mathbf{p}_n \end{bmatrix}$$

$$\mathbf{R}^* = \arg \min_{\mathbf{R}} \sum_{i=1:n} \|\mathbf{R}\mathbf{p}_i - \mathbf{q}_i\|^2 \Leftrightarrow \max tr(\mathbf{Q}^T \mathbf{R} \mathbf{P})$$

- 求 $\mathbf{R}$ 使得 $\mathbf{Q}^T \mathbf{R} \mathbf{P}$ 的迹最大
- 根据矩阵的迹的性质 $tr(\mathbf{AB})=tr(\mathbf{BA})$

$$\cdot \quad tr(\mathbf{Q}^T \mathbf{R} \mathbf{P}) = tr(\mathbf{R} \mathbf{P} \mathbf{Q}^T)$$

$$\cdot \quad \mathbf{W} = \mathbf{P} \mathbf{Q}^T \quad \mathbf{R}^* = \arg \max_{\mathbf{R}} tr(\mathbf{R} \mathbf{W}) \quad ?$$

- $\mathbf{W}$ 是一个 $d \times d$ 的矩阵
- 对 $\mathbf{W}$ 做SVD分解

$$\mathbf{W} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

$$tr(\mathbf{R} \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T) = tr(\mathbf{\Sigma} \mathbf{V}^T \mathbf{R} \mathbf{U})$$

$$\mathbf{\Sigma} = \begin{bmatrix} \delta_1 & & & \\ & \delta_2 & & \\ & & \ddots & \\ & & & \delta_d \end{bmatrix}$$

$\mathbf{V}, \mathbf{R}, \mathbf{U}$ 都是单位正交矩阵
 $\mathbf{V}^T \mathbf{R} \mathbf{U}$ 也是单位正交矩阵

- 再设

$$\mathbf{M} = \mathbf{V}^T \mathbf{R} \mathbf{U}$$

$$\mathbf{M}^T \mathbf{M} = \mathbf{I} \Rightarrow \mathbf{m}_j^T \mathbf{m}_j = \sum_{i=1}^d m_{ij}^2 = 1 \Rightarrow m_{ij} \leq 1$$

$$\text{tr}(\Sigma \mathbf{M}) = \begin{pmatrix} \sigma_1 & & & \\ & \sigma_2 & & \\ & & \ddots & \\ & & & \sigma_d \end{pmatrix} \begin{pmatrix} m_{11} & m_{12} & \dots & m_{1d} \\ m_{21} & m_{22} & \dots & m_{2d} \\ \vdots & \vdots & \ddots & \vdots \\ m_{d1} & m_{d2} & \dots & m_{dd} \end{pmatrix} = \sum_{i=1}^d \sigma_i m_{ii} \leq \sum_{i=1}^d \sigma_i$$

$$\text{tr}(\Sigma \mathbf{M})$$

$$\mathbf{V}^T \mathbf{R} \mathbf{U} = \mathbf{I}$$

$$\mathbf{V}^T \mathbf{R} \mathbf{U} = \mathbf{I} \quad \text{时取得最大值}$$

最终推导出 $\mathbf{R}^* = \mathbf{V} \mathbf{U}^T$ 是前述最优化问题的最优解

SVD分解方法总结

- 构造协方差矩阵

$$\mathbf{M} = (\mathbf{P} - \mathbf{C}_s)(\mathbf{Q} - \mathbf{C}_T)^T$$

- 对矩阵 $\mathbf{M}$ 进行SVD分解

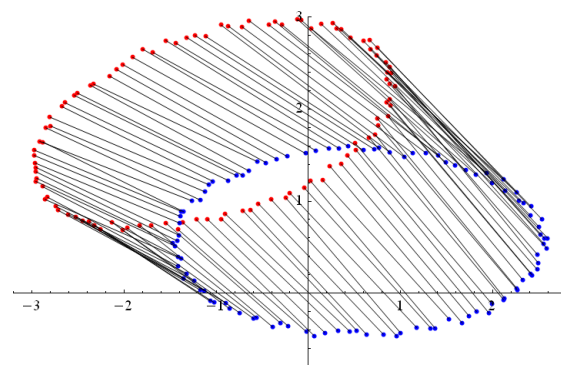
$$\mathbf{M} = \mathbf{U}\mathbf{W}\mathbf{V}^T$$

- 旋转矩阵

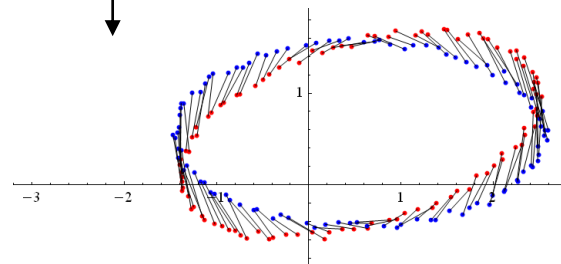
$$\mathbf{R} = \mathbf{V}\mathbf{U}^T$$

- 计算变换后的坐标

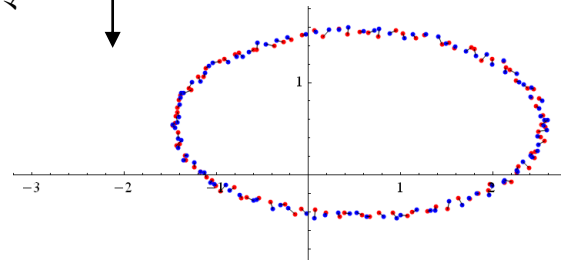
$$p'_i = c_T + \mathbf{R}(p_i - c_s)$$



平移

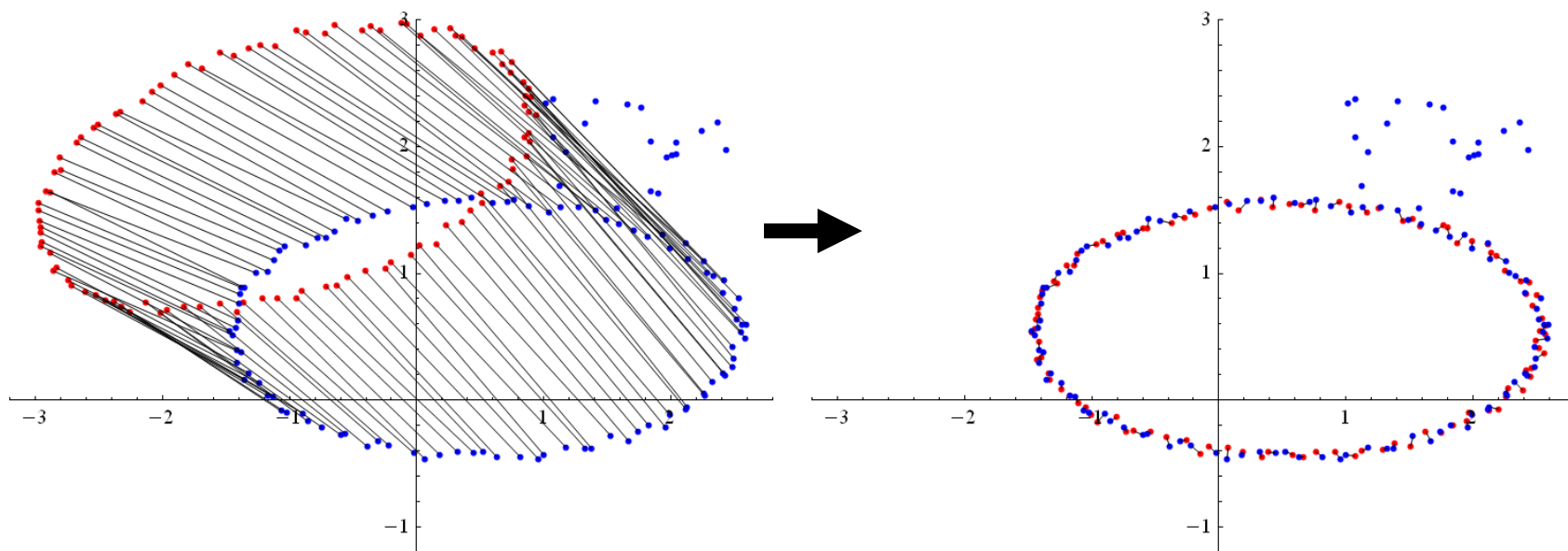


旋转



SVD分解方法的特点

- 需要两个点集之间的对应关系
- 受噪声影响小



//准备3D点

```
for (DMatch m:matches) {  
    ushort d1 = depth1.ptr<unsigned short>(int(keypoints_1[m.queryIdx].pt.y))  
    ushort d2 = depth2.ptr<unsigned short>(int(keypoints_2[m.trainIdx].pt.y))  
    if (d1 == 0 || d2 == 0)    // bad depth  
        continue;  
    Point2d p1 = pixel2cam(keypoints_1[m.queryIdx].pt, K);  
    Point2d p2 = pixel2cam(keypoints_2[m.trainIdx].pt, K);  
    float dd1 = float(d1) / 5000.0;  
    float dd2 = float(d2) / 5000.0;  
    pts1.push_back(Point3f(p1.x * dd1, p1.y * dd1, dd1));  
    pts2.push_back(Point3f(p2.x * dd2, p2.y * dd2, dd2));  
}
```

//ICP计算位姿

```
cout << "3d-3d pairs: " << pts1.size() << endl;  
Mat R, t;  
pose_estimation_3d3d(pts1, pts2, R, t);  
cout << "ICP via SVD results: " << endl;  
cout << "R = " << R << endl;  
cout << "t = " << t << endl;  
cout << "R_inv = " << R.t() << endl;  
cout << "t_inv = " << -R.t() * t << endl;
```

小结：

- 基于视觉求相机位姿变换的方法主要包括：
- 2D-2D：对极几何，8点法，单应矩阵方法
- 3D-2D：PnP，DLT方法，重投影误差最小化方法
- 3D-3D：ICP，SVD方法，对齐误差最小化方法



光流法 (Optical Flow)

王永才

前述特征点法的问题

- 回忆特征点法VO的做法

- 在图像中提取特征点并计算特征描述
- 在不同图像中寻找特征匹配
- 利用匹配点信息计算相机位姿

非常耗时 ~10+ms in ORB

非常耗时 $O(n^2)$ in brute force matching

比较快速 < 1ms

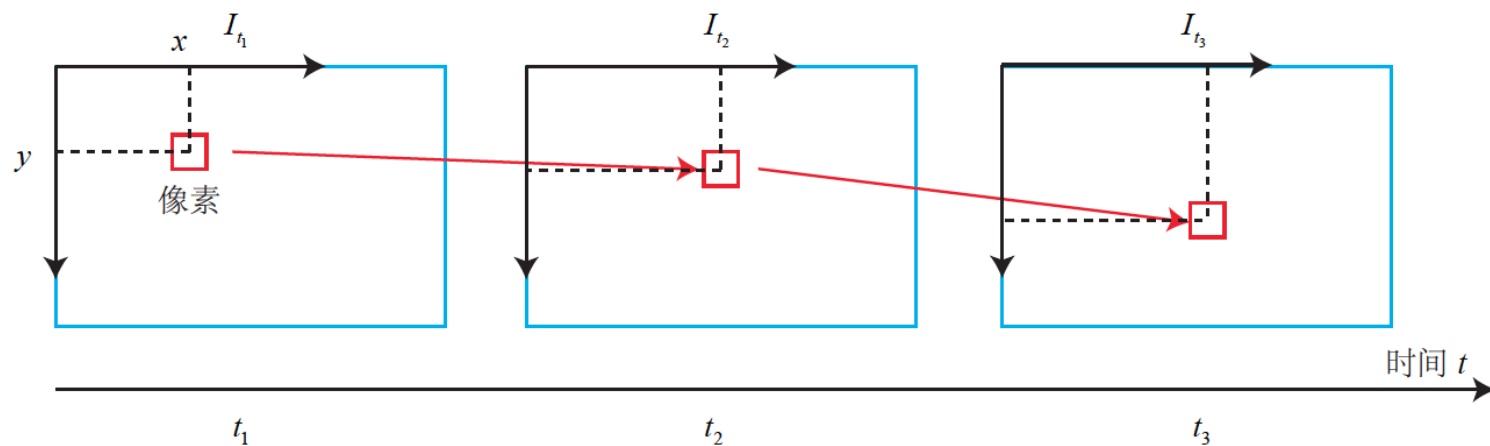
- 能否不使用特征匹配计算VO？

光流和直接法

- 不提特征计算VO的思路：
 - 通过其他方式寻找配对点
- 光流
 - 无配对点
- 直接法

光流

- 一般分为稀疏光流和稠密光流
 - 稀疏以Lucas-Kanade (LK) 光流为代表
 - 稠密以Horn-Schunck (HS) 光流为代表
 - 本质上是估计像素在不同时刻图像中的运动



灰度不变假设: $I(x_1, y_1, t_1) = I(x_2, y_2, t_2) = I(x_3, y_3, t_3)$

图 8-1 LK 光流法示意图。

光流

- 设 t 时刻位于 x, y 处像素点的灰度值为 $I(x, y, t)$.
- 在 $t+dt$ 时刻，该像素运动到了 $I(x + dx, y + dy, t + dt)$
- 希望计算运动 dx, dy
- 灰度不变假设： $I(x + dx, y + dy, t + dt) = I(x, y, t)$.
- 注意：灰度不变是一种理想的假设，实际当中由于高光/阴影/材质/曝光等不同，很可能不成立。

光流

$$I(x + dx, y + dy, t + dt) = I(x, y, t).$$

- 对 $t+dt$ 时刻的灰度进行Taylor展开并保留一阶项：

$$I(x + dx, y + dy, t + dt) \approx I(x, y, t) + \frac{\partial I}{\partial x} dx + \frac{\partial I}{\partial y} dy + \frac{\partial I}{\partial t} dt.$$

- 由于灰度不变，所以

$$\frac{\partial I}{\partial x} dx + \frac{\partial I}{\partial y} dy + \frac{\partial I}{\partial t} dt = 0.$$

$$\text{因此} \Rightarrow \frac{\partial I}{\partial x} \frac{dx}{dt} + \frac{\partial I}{\partial y} \frac{dy}{dt} = -\frac{\partial I}{\partial t}.$$

x方向梯度

y方向梯度

随时间变化

希望求解 dx/dt , dy/dt

$$\frac{\partial I}{\partial x} \frac{dx}{dt} + \frac{\partial I}{\partial y} \frac{dy}{dt} = -\frac{\partial I}{\partial t}.$$

- 但本式是一个二元一次线性方程，欠定
- 需要引用额外的约束
- 假定一个窗口（ $w \times w$ ）内光度不变：
- 通过超定最小二乘解求得运动 u, v

$$\begin{bmatrix} I_x & I_y \end{bmatrix}_k \begin{bmatrix} u \\ v \end{bmatrix} = -I_{tk}, \quad k = 1, \dots, w^2.$$

$$A = \begin{bmatrix} [I_x, I_y]_1 \\ \vdots \\ [I_x, I_y]_k \end{bmatrix}, b = \begin{bmatrix} I_{t1} \\ \vdots \\ I_{tk} \end{bmatrix}. \quad \begin{bmatrix} u \\ v \end{bmatrix}^* = -(A^T A)^{-1} A^T b.$$

光流

- 注解：
 - 可以看成最小化像素误差的非线性优化
 - 每次使用了Taylor一阶近似，在离优化点较远时效果不佳，往往需要迭代多次
 - 运动较大时要使用金字塔
 - 可以用于跟踪图像中的稀疏关键点的运动轨迹



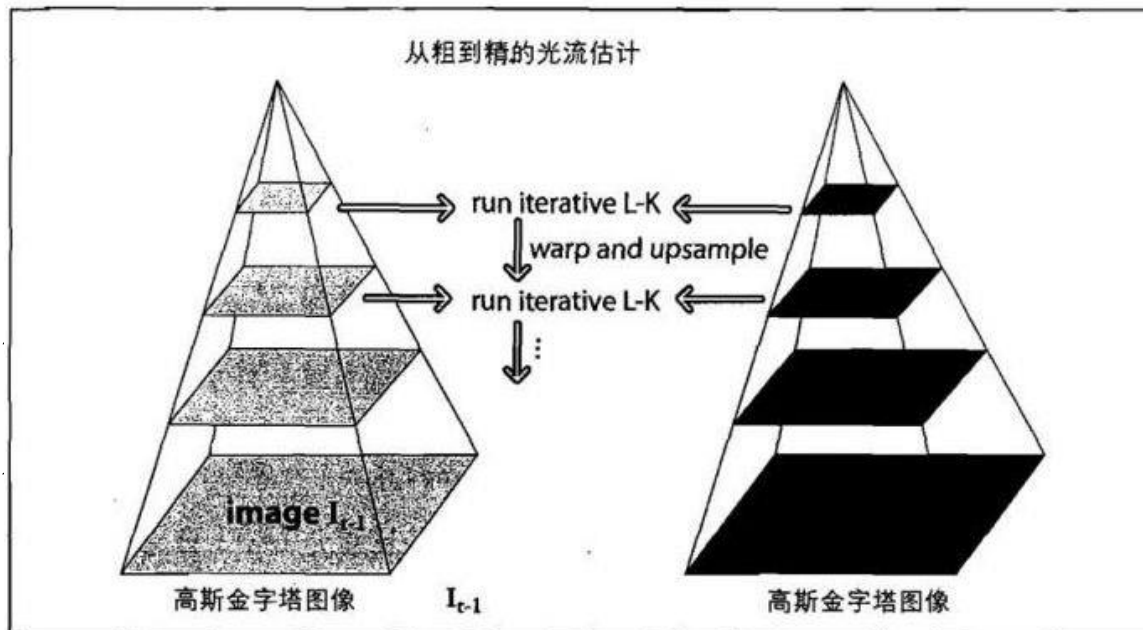
实践：LK光流

```
// key points, using GFTT here.
vector<KeyPoint> kp1;
Ptr<GFTTDetector> detector = GFTTDetector::create(500, 0.01, 20);
detector->detect(img1, kp1);

// use opencv's flow for validation
vector<Point2f> pt1, pt2;
for (auto &kp: kp1) pt1.push_back(kp.pt);
vector<uchar> status;
vector<float> error;
t1 = chrono::steady_clock::now();
cv::calcOpticalFlowPyrLK(img1, img2, pt1, pt2, status, error);
t2 = chrono::steady_clock::now();
time_used = chrono::duration_cast<chrono::duration<double>>(t2 - t1);
cout << "optical flow by opencv: " << time_used.count() << endl;
```

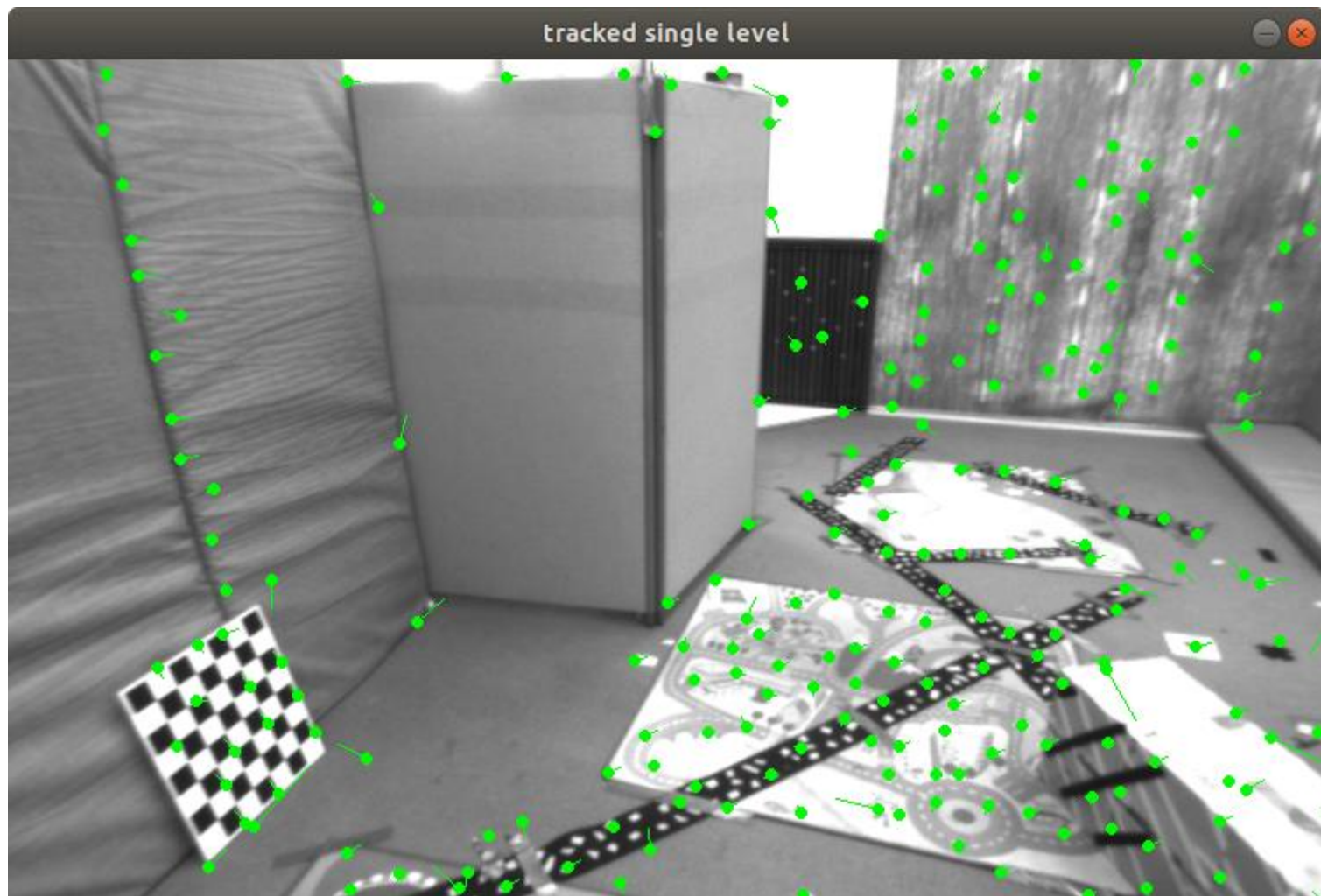
多层光流

- 单层光流的问题
 - 初始值接近最优
 - 运动较快时易陷
- 多层光流

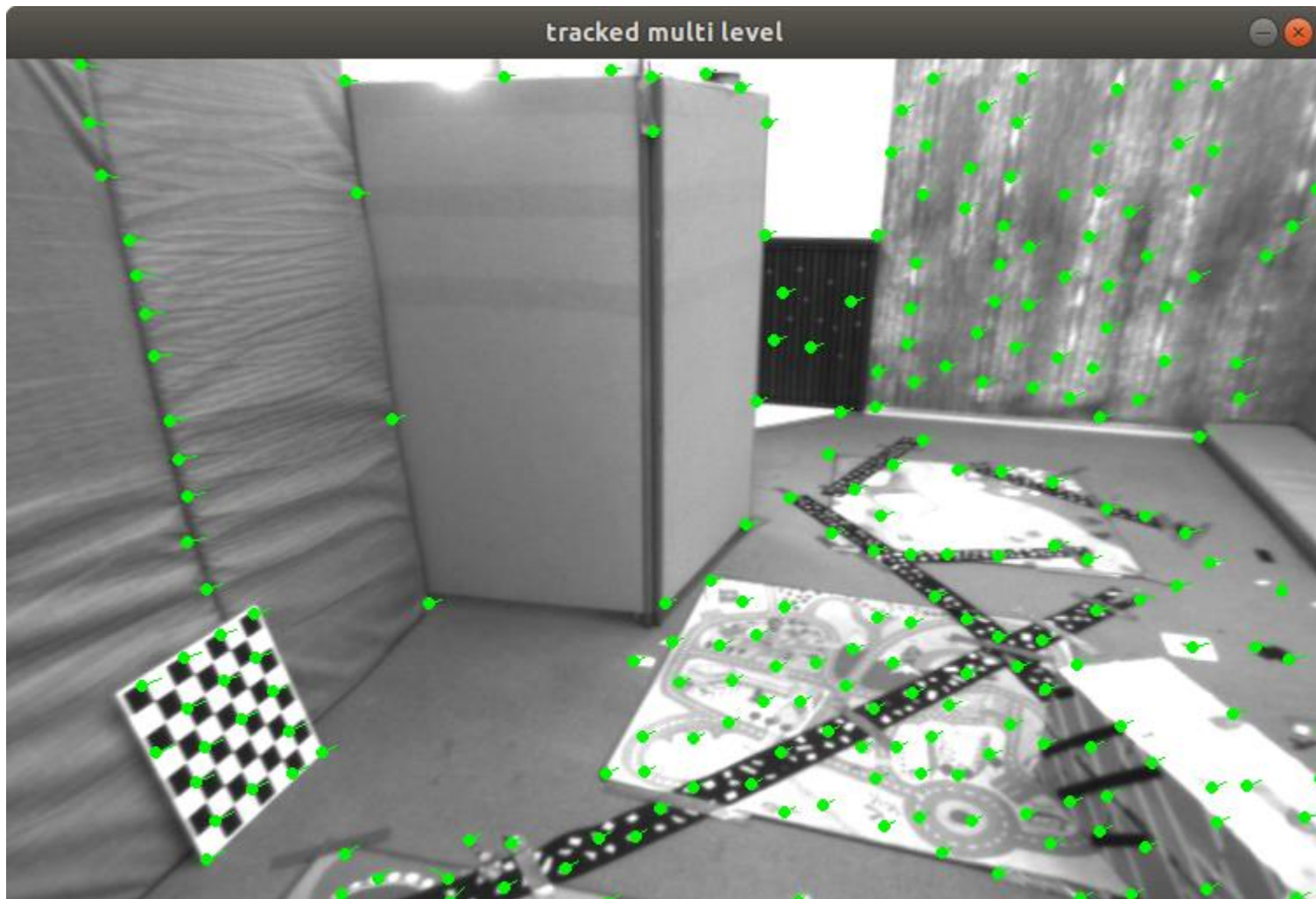


- 图像金字塔，对图像进行尺度缩放
- 计算光流时，从顶层开始计算
- 把上一层的追踪结果，作为下一层的初始值。

单层光流给出的特征点匹配



多层光流给出的特征点匹配



小结

- 2D-2D: 对极几何, 8点法, 单应矩阵方法
- 3D-2D: PnP, DLT方法, 重投影误差最小化方法
- 3D-3D: ICP, SVD方法, 对齐误差最小化方法
- 2D-2D: 用光流找对应点, 用对极几何算位姿变换
- 3D-2D: 用直接法, 直接用重投影误差最小化方法, 计算对应点和变换矩阵



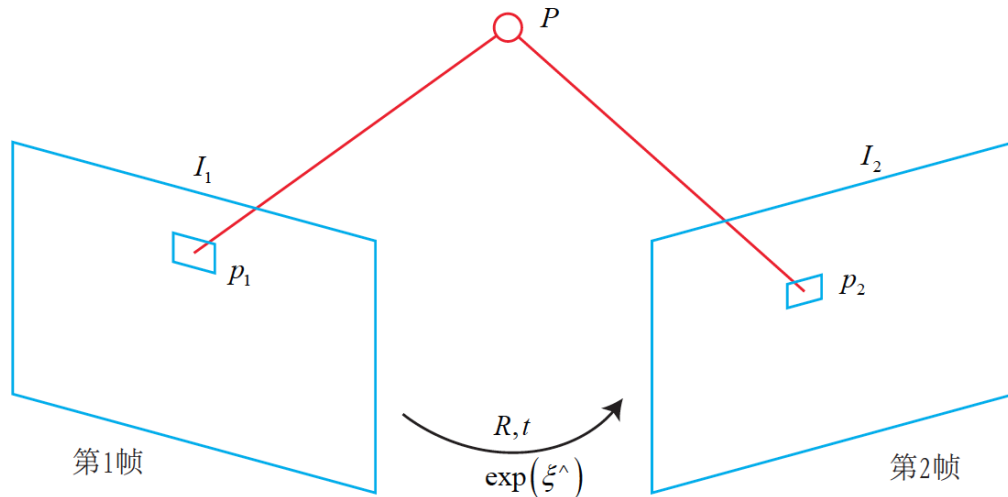
直接法 (Direct Visual Odometry)

王永才

直接法

- 光流仅估计了像素间的平移，但
 - 没有用到相机本身的几何结构
 - 没有考虑到相机的旋转和图像的缩放
 - 直接法则考虑了这些信息

直接法



直接法的推导

假设有两个帧，运动未知，但有初始估计 R, t ，并且知道 P

第1帧上看到了点 P ，投影为 p_1

按照初始估计， P 在第2帧上投影为 p_2

投影关系：

$$p_1 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_1 = \frac{1}{Z_1} K P,$$
$$p_2 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_2 = \frac{1}{Z_2} K (R P + t) = \frac{1}{Z_2} K (\exp(\xi^\wedge) P)_{1:3}.$$

直接法

- 同时求由相机1到相机2的位姿变化和特征点的对应关系

$$p_1 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_1 = \frac{1}{Z_1} K P,$$

$$p_2 = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}_2 = \frac{1}{Z_2} K (R P + t) = \frac{1}{Z_2} K (\exp(\xi^\wedge) P)_{1:3}.$$

我们实际上并不知道对应关系

- 为了估计相机的运动，建立光度误差最小化问题

最小化光度误差： $e = I_1(p_1) - I_2(p_2)$ 。

$$\min_{\xi} J(\xi) = \sum_{i=1}^N e_i^T e_i, \quad e_i = I_1(p_{1,i}) - I_2(p_{2,i}).$$

- 在相机1随机产生很多非边缘点，在相机2找对应最小光度误差点。

- 找到对应点的同时计算出相机的位姿变换矩阵。

- 待估计的量为相机运动 ξ ，我们关心误差相对于相机运动的导数

$$\begin{aligned}
e(\xi \oplus \delta\xi) &= I_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - I_2 \left(\frac{1}{Z_2} \mathbf{K} \exp(\delta\xi^\wedge) \exp(\xi^\wedge) \mathbf{P} \right) \\
&\approx I_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - I_2 \left(\frac{1}{Z_2} \mathbf{K} (1 + \delta\xi^\wedge) \exp(\xi^\wedge) \mathbf{P} \right) \\
&= I_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - I_2 \left(\frac{1}{Z_2} \mathbf{K} \exp(\xi^\wedge) \mathbf{P} + \frac{1}{Z_2} \mathbf{K} \delta\xi^\wedge \exp(\xi^\wedge) \mathbf{P} \right).
\end{aligned}$$

类似于PnP, 记

$$\mathbf{q} = \delta\xi^\wedge \exp(\xi^\wedge) \mathbf{P},$$

$$\mathbf{u} = \frac{1}{Z_2} \mathbf{K} \mathbf{q}.$$

这里的 $\mathbf{q}$ 为 $\mathbf{P}$ 在扰动之后, 位于第二个相机坐标系下的坐标, 而 $\mathbf{u}$ 为它的像素坐标

利用一阶泰勒展开，有：

$$\begin{aligned}
 e(\boldsymbol{\xi} \oplus \delta\boldsymbol{\xi}) &= \mathbf{I}_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - \mathbf{I}_2 \left(\frac{1}{Z_2} \mathbf{K} \exp(\boldsymbol{\xi}^\wedge) \mathbf{P} + \mathbf{u} \right) \\
 &\approx \mathbf{I}_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - \mathbf{I}_2 \left(\frac{1}{Z_2} \mathbf{K} \exp(\boldsymbol{\xi}^\wedge) \mathbf{P} \right) - \frac{\partial \mathbf{I}_2}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{q}} \frac{\partial \mathbf{q}}{\partial \delta\boldsymbol{\xi}} \delta\boldsymbol{\xi} \\
 &= e(\boldsymbol{\xi}) - \frac{\partial \mathbf{I}_2}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \mathbf{q}} \frac{\partial \mathbf{q}}{\partial \delta\boldsymbol{\xi}} \delta\boldsymbol{\xi}.
 \end{aligned}$$

$$\text{其中 } e(\boldsymbol{\xi}) = \mathbf{I}_1 \left(\frac{1}{Z_1} \mathbf{K} \mathbf{P} \right) - \mathbf{I}_2 \left(\frac{1}{Z_2} \mathbf{K} \exp(\boldsymbol{\xi}^\wedge) \mathbf{P} \right)$$

我们看到，一阶导数由于链式法则分成了三项，而这三项都是容易计算的：

1. $\partial \mathbf{I}_2 / \partial \mathbf{u}$ 为 $\mathbf{u}$ 处的像素梯度；
2. $\partial \mathbf{u} / \partial \mathbf{q}$ 为投影方程关于相机坐标系下的三维点的导数。记 $\mathbf{q} = [X, Y, Z]^T$ ，根据上一节的推导，导数为：

$$\frac{\partial \mathbf{u}}{\partial \mathbf{q}} = \begin{bmatrix} \frac{\partial u}{\partial X} & \frac{\partial u}{\partial Y} & \frac{\partial u}{\partial Z} \\ \frac{\partial v}{\partial X} & \frac{\partial v}{\partial Y} & \frac{\partial v}{\partial Z} \end{bmatrix} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} \end{bmatrix}$$

3. $\partial \mathbf{q} / \partial \delta \boldsymbol{\xi}$ 为变换后的三维点对变换的导数，这在李代数章节已经介绍过了：

$$\frac{\partial \mathbf{q}}{\partial \delta \boldsymbol{\xi}} = [\mathbf{I}, -\mathbf{q}^{\wedge}]$$

在实践中，由于后两项只与三维点 $\mathbf{q}$ 有关，而与图像无关，我们经常把它合并在一起：

$$\frac{\partial \mathbf{u}}{\partial \delta \boldsymbol{\xi}} = \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} & -\frac{f_x XY}{Z^2} & f_x + \frac{f_x X^2}{Z^2} & -\frac{f_x Y}{Z} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} & -f_y - \frac{f_y Y^2}{Z^2} & \frac{f_y XY}{Z^2} & \frac{f_y X}{Z} \end{bmatrix}$$

于是，我们推导了误差相对于李代数的雅可比矩阵：

$$\mathbf{J} = -\frac{\partial \mathbf{I}_2}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \delta \boldsymbol{\xi}}.$$

对于 N 个点的问题，我们可以用这种方法**计算优化问题的雅可比**，然后使用 G-N 或 L-M 计算增量，迭代求解。

直接法中

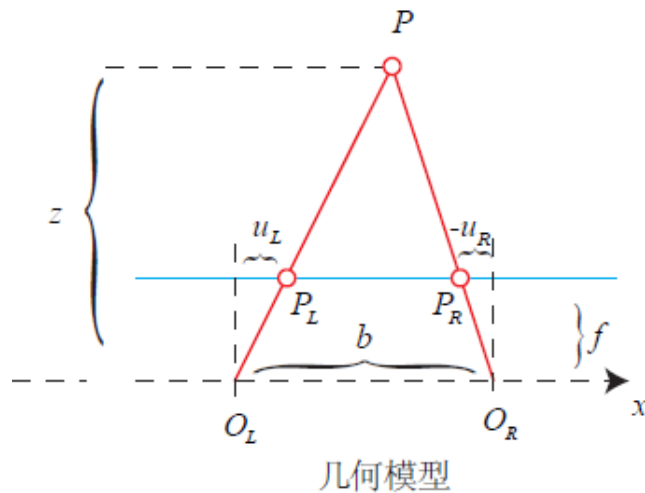
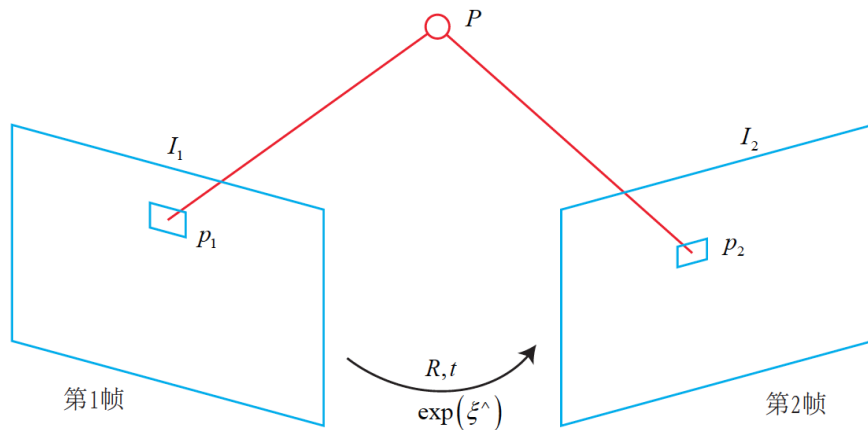
在相机1中选择的点的3D空间位置怎么来？

1. 在RGB-D中，任意像素的3D位置是可以直接获得的。

2. 在双目相机中，可以根据视差来获得像素深度。

$$z = \frac{fb}{d}, \quad d = u_L - u_R.$$

3. 单目相机中，就比较困难，还必须考虑深度带来的不确定性，第13章中详细介绍。



直接法的分类

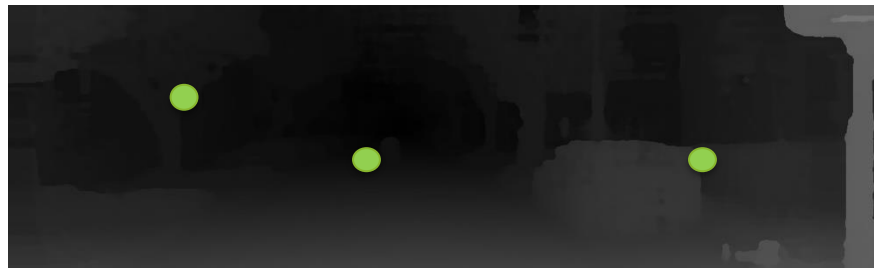
根据 P 的来源，可以把直接法分为三类：

1. P 来自于稀疏关键点，我们称之为**稀疏直接法**。通常我们使用数百个至上千个关键点，这种稀疏直接法不必计算描述子，并且只使用数百个像素，因此速度最快，但只能计算稀疏的重构。
2. P 来自部分像素。我们看到在导数公式中，如果像素梯度为零，整一项雅可比就为零，不会对计算运动增量有任何贡献。因此，可以考虑**只使用带有梯度的像素点，舍弃像素梯度不明显的地方。这称之为半稠密 (Semi-Dense) 的直接法**，可以重构一个半稠密结构。
3. P 为所有像素，称为**稠密直接法**。稠密重构需要计算所有像素(一般几十万至几百万个)，因此多数不能在现有的 CPU 上实时计算，需要 GPU 的加速。但是，如前面所讨论的，梯度不明显的点，在运动估计中不会有太大贡献，在重构时也会难以估计位置。

直接法的实验



当前时刻左目相机图像



双目视差

1. 根据双目视差，可以首先求出当前图像中采样点的空间位置

$$z = \frac{fb}{d}, \quad d = u_L - u_R, \quad x = (u - c_x)z/f_x, \quad y = (v - c_y)z/f_y$$

2. 在如下图像中通过像素差最小化，找出对应点并计算相机的位姿变换矩阵



直接计算出相机位姿变换矩阵

$$\min_{\xi} J(\xi) = \sum_{i=1}^N e_i^T e_i, \quad e_i = \mathbf{I}_1(p_{1,i}) - \mathbf{I}_2(p_{2,i}).$$

直接法算法1

$$\min_{\xi} J(\xi) = \sum_{i=1}^N e_i^T e_i, \quad e_i = I_1(p_{1,i}) - I_2(p_{2,i}).$$

for iter=0; iter<iterations; iter ++

for i=1... n //基于所有点的测量求最优增量

$$e_i = I_1 \left(\frac{1}{Z_1} K P_i \right) - I_2 \left(\frac{1}{Z_2} K \exp(\xi^\wedge) P_i \right)$$

$$J_i = -\frac{\partial I_2}{\partial \mathbf{u}} \begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} & -\frac{f_x XY}{Z^2} & f_x + \frac{f_x X^2}{Z^2} & -\frac{f_x Y}{Z} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} & -f_y - \frac{f_y Y^2}{Z^2} & \frac{f_y XY}{Z^2} & \frac{f_y X}{Z} \end{bmatrix}$$

$$H+ = J_i^T J_i$$

$$b+ = J_i^T e_i$$

$$\text{cost} + = e_i^* e_i$$

endfor

$$\Delta \xi = -H^{-1} b$$

//将最优增量应用在估计的变换矩阵上

$$T_{est} = \exp(\Delta \xi) T_{est}$$

//如果收敛或者达到最大迭代次数则退出

If cost > lastcost break;

If Converge($\Delta \xi$) break;

endfor

slambook2/ch8/direct
_method.cpp

调用G2O实现直接法

在使用 g2o 之前，需要把直接法抽象成一个图优化问题。

直接法是由以下顶点和边组成的：

- 优化变量为一个**相机位姿**，因此**需要一个位姿顶点**。由于我们在推导中使用了李代数，故程序中使用李代数表达的 $SE(3)$ 位姿顶点。使用“VertexSE3Expmap”作为相机位姿。
- 误差项为**单个像素的光度误差**。由于整个优化过程中 $I_1(p_1)$ 保持不变，我们可以把它当成一个固定的预设值，然后**调整相机位姿，使 $I_2(p_2)$ 接近这个值**。于是，这种边只连接一个顶点，为一元边。
- 由于 g2o 中本身没有计算光度误差的边，**需要自己定义一种新的边**。

直接法的图优化求解分析

直接法图优化问题是由一个相机位姿顶点与许多条一元边组成的。

如果使用稀疏的直接法，那我们大约会有几百至几千条这样的边；稠密直接法则会有几十万条边。

优化问题对应的线性方程是计算李代数增量，本身规模不大(6×6)。

主要的 计算时间会花费在每条边的误差与雅可比的计算上。

定义直接法的光度误差边

任务是：

1. 继承 `BaseUnaryEdge` 类
2. 实现它的两个虚函数：`computeError()`, `linearizeOpplus()`

//继承自 `g2o::BaseUnaryEdge`

//类的成员包括：(1)世界坐标的一个点；(2)相机内参矩阵；(3)参考图像

```
class EdgeSE3ProjectDirect: public BaseUnaryEdge< 1, double, VertexSE3Expmap>
{
public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW

    EdgeSE3ProjectDirect() {}

    EdgeSE3ProjectDirect ( Eigen::Vector3d point, float fx, float fy, float cx, float cy, cv::Mat*
image )
: x_world_ ( point ), fx_ ( fx ), fy_ ( fy ), cx_ ( cx ), cy_ ( cy ), image_ ( image )
{}
}
```

```

virtual void computeError()
{
    const VertexSE3Expmap* v = static_cast<const VertexSE3Expmap*> ( _vertices[0] );

    Eigen::Vector3d x_local = v->estimate().map ( x_world_ );
    //获得x_world_ 的相机坐标系坐标

    float x = x_local[0]*fx_/x_local[2] + cx_;
    float y = x_local[1]*fy_/x_local[2] + cy_;
    //获得x_world_的相平面投影坐标

    // check x,y is in the image
    if ( x-4<0 || ( x+4 ) >image_->cols || ( y-4 ) <0 || ( y+4 ) >image_->rows )
    {
        _error ( 0,0 ) = 0.0;
        this->setLevel ( 1 );
    }
    else
    {
        _error ( 0,0 ) = getPixelValue ( x,y ) - _measurement;
        //计算光度误差
    }
}
}

```

```

virtual void linearizeOplus( )
{
    if ( level() == 1
        _jacobianOplusXi = Eigen::Matrix<double, 1, 6>::Zero(); return;
    VertexSE3Expmap* vtx = static_cast<VertexSE3Expmap*> ( _vertices[0] );
    Eigen::Vector3d xyz_trans = vtx->estimate().map ( x_world_ ); // q in book

    double x = xyz_trans[0];
    double y = xyz_trans[1];
    double invz = 1.0/xyz_trans[2];
    double invz_2 = invz*invz;
    float u = x*fx_*invz + cx_;
    float v = y*fy_*invz + cy_;
    Eigen::Matrix<double, 2, 6> jacobian_uv_ksai;

    jacobian_uv_ksai ( 0,0 ) = - x*y*invz_2 *fx_;
    jacobian_uv_ksai ( 0,1 ) = ( 1+ ( x*x*invz_2 ) ) *fx_;
    jacobian_uv_ksai ( 0,2 ) = - y*invz *fx_;
    jacobian_uv_ksai ( 0,3 ) = invz *fx_;
    jacobian_uv_ksai ( 0,4 ) = 0;
    jacobian_uv_ksai ( 0,5 ) = -x*invz_2 *fx_;

    jacobian_uv_ksai ( 1,0 ) = - ( 1+y*y*invz_2 ) *fy_;
    jacobian_uv_ksai ( 1,1 ) = x*y*invz_2 *fy_;
    jacobian_uv_ksai ( 1,2 ) = x*invz *fy_;
    jacobian_uv_ksai ( 1,3 ) = 0;
    jacobian_uv_ksai ( 1,4 ) = invz *fy_;
    jacobian_uv_ksai ( 1,5 ) = -y*invz_2 *fy_;

    Eigen::Matrix<double, 1, 2> jacobian_pixel_uv;

    jacobian_pixel_uv ( 0,0 ) = ( getPixelValue ( u+1,v )-getPixelValue ( u-1,v ) ) /2;
    jacobian_pixel_uv ( 0,1 ) = ( getPixelValue ( u,v+1 )-getPixelValue ( u,v-1 ) ) /2;

    _jacobianOplusXi = jacobian_pixel_uv*jacobian_uv_ksai;
}

```

$$\begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} & -\frac{f_x XY}{Z^2} & f_x + \frac{f_x X^2}{Z^2} & -\frac{f_x Y}{Z} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} & -f_y - \frac{f_y Y^2}{Z^2} & \frac{f_y XY}{Z^2} & \frac{f_y X}{Z} \end{bmatrix}$$

$$\frac{\partial I_2}{\partial \mathbf{u}}$$

$$\mathbf{J} = -\frac{\partial I_2}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial \delta \boldsymbol{\xi}}.$$

```

int main ( int argc, char** argv )
{
    srand ( ( unsigned int ) time ( 0 ) );
    string path_to_dataset = argv[1];
    string associate_file = path_to_dataset + "/associate.txt";
    ifstream fin ( associate_file );
    string rgb_file, depth_file, time_rgb, time_depth;
    cv::Mat color, depth, gray;
    vector<Measurement> measurements;
    // 相机内参
    Eigen::Matrix3f K;
    K<<fx,0.f,cx,0.f,fy,cy,0.f,0.f,1.0f;
    Eigen::Isometry3d Tcw = Eigen::Isometry3d::Identity();
    cv::Mat prev_color;
    for ( int index=0; index<10; index++ )
    {
        cout<<"***** loop "<<index<<" *****"<<endl;
        fin>>time_rgb>>rgb_file>>time_depth>>depth_file;
        color = cv::imread ( path_to_dataset+"/"+rgb_file );
        depth = cv::imread ( path_to_dataset+"/"+depth_file, -1 );
        cv::cvtColor ( color, gray, cv::COLOR_BGR2GRAY );
        if ( index ==0 )
        {
            // 对第一帧提取FAST特征点
            vector<cv::KeyPoint> keypoints;
            cv::Ptr<cv::FastFeatureDetector> detector = cv::FastFeatureDetector::create();
            detector->detect ( color, keypoints );
            for ( auto kp:keypoints )
            {
                Eigen::Vector3d p3d = project2Dto3D ( kp.pt.x, kp.pt.y, d, fx, fy, cx, cy, depth_scale );
                float grayscale = float ( gray.ptr<uchar> ( cvRound ( kp.pt.y ) ) [ cvRound ( kp.pt.x ) ] );
                measurements.push_back ( Measurement ( p3d, grayscale ) );
            }
            prev_color = color.clone();
            continue;
        }
        poseEstimationDirect ( measurements, &gray, K, Tcw );
        cout<<"Tcw="<<Tcw.matrix() <<endl;
    }
}

```

实践直接法

直接法的直观解释

- 像素灰度引导着优化的方向
- 要使优化成立，必须保证从初始估计到最优估计中间的梯度一直下降
- 这很容易受到图像非凸性的影响（可部分地由金字塔减轻）

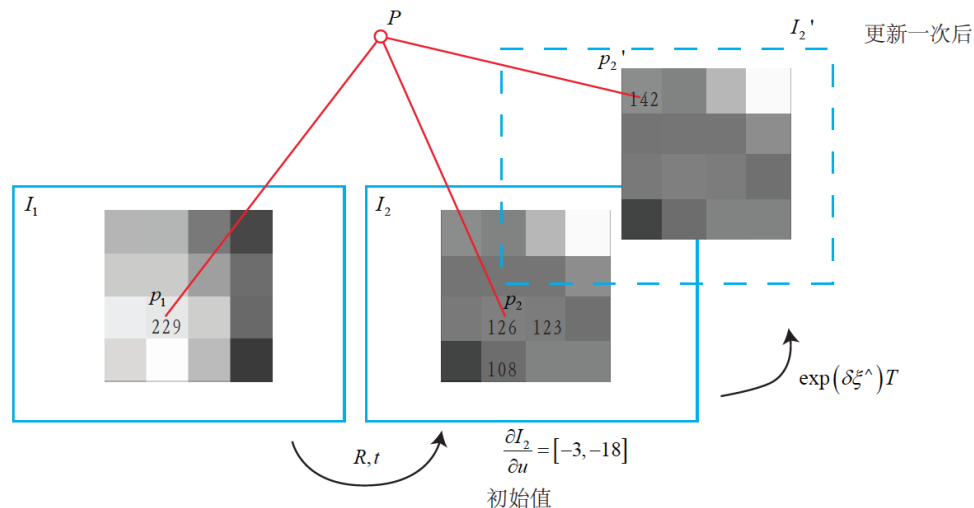
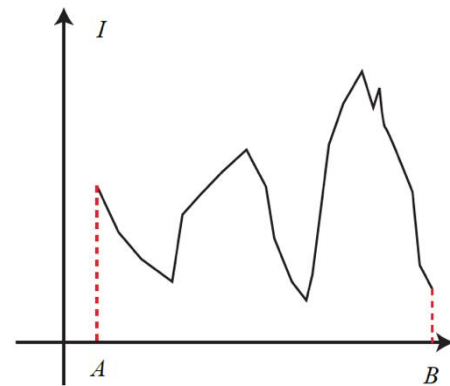
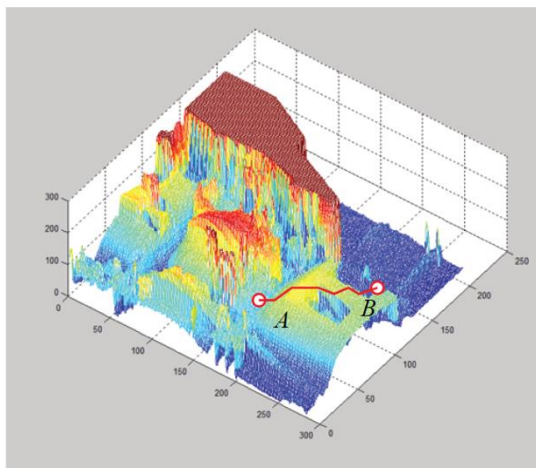


图 8-6 一次迭代的图形化显示。



直接法总结

- 可以看到，直接法的雅可比项有一个图像梯度因子
 - 因此，在图像梯度不明显的地方，对相机运动估计的贡献就小
- 根据使用的图像信息不同，可分为：
 - 稀疏直接法：只处理稀疏角点或关键点
 - 稠密直接法：使用所有像素
 - 半稠密直接法：使用部分梯度明显的像素
- 优势
 - 省略特征提取的时间
 - 只需有像素梯度而不必是角点（对白墙等地方有较好效果）
 - 可稠密或半稠密
- 劣势
 - 灰度不变难以满足（易受曝光和模糊影响）
 - 单像素区分性差
 - 图像非凸性

小结

- 2D-2D: 对极几何, 8点法, 单应矩阵方法
- 3D-2D: PnP, DLT方法, 重投影误差最小化方法
- 3D-3D: ICP, SVD方法, 对齐误差最小化方法
- 2D-2D: 用光流找对应点, 用对极几何算位姿变换
- 3D-2D: 用直接法, 直接用重投影误差最小化方法, 计算对应点和变换矩阵